inference-engine

An LLM serving engine built from scratch:
paged KV cache, continuous batching, speculative decoding

Deep Dive

How to read this document

This is an exhaustive walkthrough of one project, written for a reader who has never served a language model and does not need to have. Every technical term is defined in a **Jargon** box the first time it appears. Every red **honest** box is a real finding about the code: a weakness, an inconsistency, a claim that does not survive contact with the source, or a limit worth knowing before someone else finds it. They are the most valuable part of the document.

Contents

1 What this project is

`inference-engine` is a program that takes a trained language model and turns it into a *service*: an HTTP server that many people can send prompts to at the same time, that streams words back as they are produced, and that shares one machine's memory between all of them without falling over.

It is written in Python on top of PyTorch. It runs the GPT-2 family of models (`gpt2` and `distilgpt2`) on an ordinary CPU. Hugging Face, the standard library for pretrained models, is used for exactly two things: to download the trained weights, and to turn text into numbers and back. Every mechanism that makes it a *serving engine*, the forward pass, the attention computation, the memory allocator, the scheduler, the sampler, the speculative decoder and the HTTP surface, is implemented in this repository.

The one-sentence summary

A language model can only produce one word at a time, and remembering what it has already read is what makes that affordable. This project is an implementation of the memory management and scheduling that makes remembering affordable *for many users at once*, rebuilt small enough to fit in one person's head and measured honestly instead of assumed.

1.1 What it is not

Being precise about the boundary matters more than the feature list:

- It is **not a training system**. It never computes a gradient and never updates a weight. It only runs models forward.
- It is **not a competitor to vLLM or TensorRT-LLM**. Those are production systems with hand-written GPU kernels. This is a study implementation of the same ideas, at roughly 1,000 lines of engine and server code, that happens to work.
- It is **not GPU code**. It runs on CPU, which changes which optimizations pay off. The project is unusually honest about this, and section 13.4 shows a headline feature (speculative decoding) measured going *backwards* on CPU, reported as such rather than quietly dropped.

1.2 The size of the thing

Area	Lines	What lives there
`engine/`	815	the whole engine: cache, scheduler, model, sampling
`server/`	206	FastAPI surface and the CLI entry point
`tests/`	731	7 test modules, 55 tests
`benchmarks/`	271	the measurement harness
`docs/`, README	355	prose documentation

Table 1: Tracked source, counted with `wc -l` over `git ls-files`. The engine is smaller than its tests, which is the right ratio for code whose whole claim is correctness under pressure.

2 Background from zero: what a language model actually does

Nothing in this section is specific to the project. It is the minimum needed for the rest to mean anything. A reader who already knows what a KV cache is can jump to section 3.

2.1 Text becomes numbers

Jargon: Token

A token is a chunk of text, usually a common word or a fragment of one. Models do not read letters or words; they read tokens. The sentence `The meaning of life is` becomes five tokens; a rare word like `antidisestablishmentarianism` might become six. Each distinct token has an integer id. GPT-2's vocabulary has 50,257 of them.

Jargon: Tokenizer

The program that converts text to token ids and back. GPT-2 uses *byte-level BPE* (Byte Pair Encoding), which means it operates on raw bytes, not characters. This detail causes a real bug later (section 9.3), so it is worth holding onto: a single character that takes several bytes to write, an emoji or an accented letter, can be *split across two tokens*.

Jargon: Logits

Given the tokens so far, the model outputs one number per vocabulary entry scoring how good that token would be as the next one. That vector of 50,257 numbers is called the logits. It is not yet a probability: turning it into one requires the softmax function, which exponentiates every entry and divides by the total so they sum to 1.

2.2 The autoregressive loop

A language model has exactly one skill: given a sequence of tokens, predict the next one. Everything else is a loop around that skill. To produce a sentence, you predict one token, glue it onto the end of the input, and predict again.

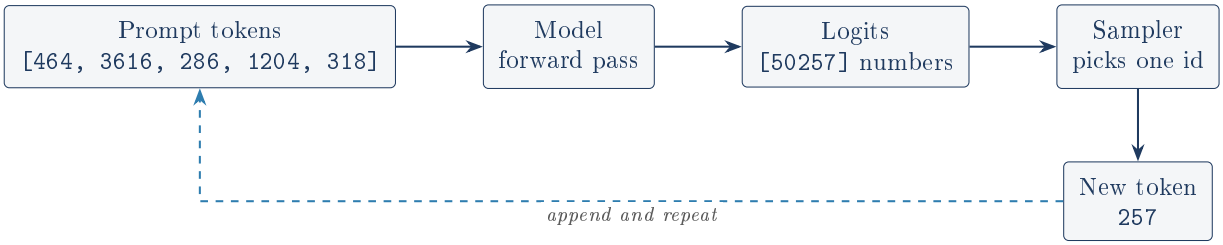


Figure 1: The autoregressive loop. Every token you read from a chatbot cost one full pass through the model. This loop is the entire job; the rest of the project exists to make it cheap and to run many of these loops at once.

2.3 Attention, and why the loop is expensive

Jargon: Transformer, layer, attention

A transformer is the model architecture behind essentially every modern language model. It is a stack of near-identical *layers*. GPT-2 has 12; distilgpt2 has 6. The important part of each layer is *attention*: for every token position, the model looks back at every earlier position and decides how much each one matters right now.

Jargon: Query, key, value (Q, K, V)

Attention works by giving every token three derived vectors. The **query** is what this position is looking for. The **key** is what a position offers, used for matching. The **value** is the

information actually retrieved. Position i compares its query against the key of every position $j \leq i$, converts those match scores to weights with softmax, and returns the weighted mixture of those positions' values. Restricting to $j \leq i$ is called *causal* masking: a token may not see the future.

Here is the problem. To predict token 501, attention at every layer needs the keys and values of tokens 1 through 500. Recomputing them means re-running the whole model over 500 positions to produce a single new token, and doing it again for token 502. Generating n tokens naively costs $O(n^2)$ work.

2.4 The KV cache, the idea the whole project rests on

Why a KV cache exists

The key and value vectors for token 5 depend only on tokens 1 through 5. They do not depend on tokens 6, 7, or 500. Causal masking guarantees the past cannot see the future, so **the past never changes**. Compute each token's keys and values once, store them, and every later step just reads them back. The loop drops from $O(n^2)$ to $O(n)$ and generation becomes practical.
That store is the **KV cache**. Everything difficult about serving language models follows from one fact: the cache is large, it grows with every token, and you have to decide who gets to keep theirs.

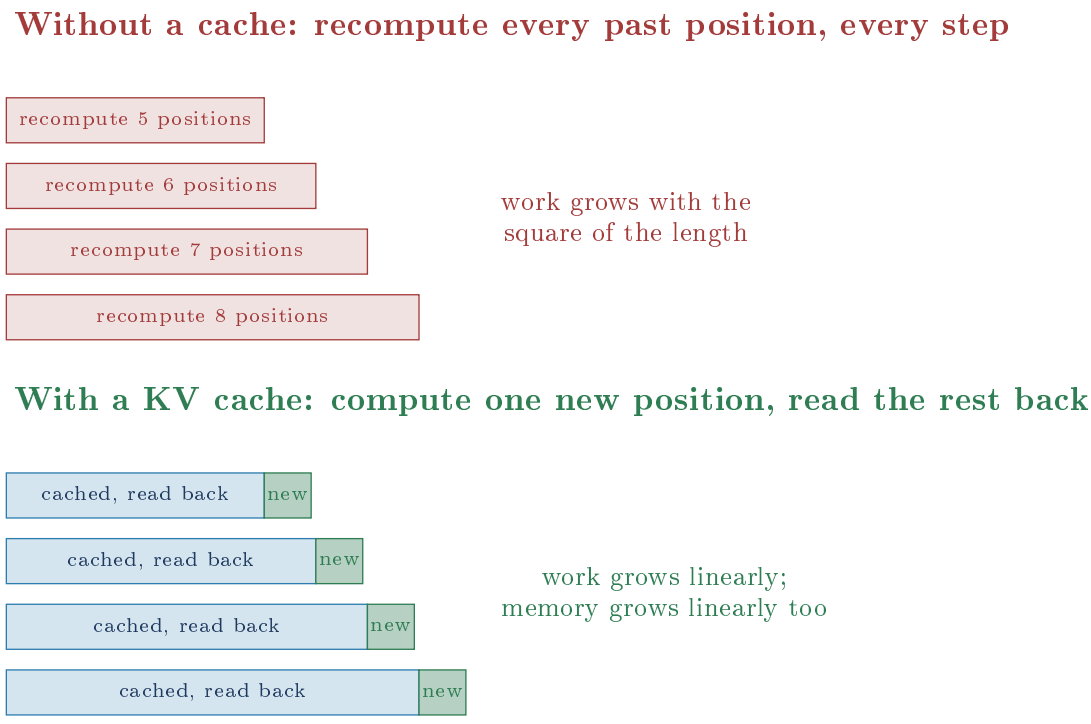


Figure 2: The trade the KV cache makes: it converts repeated computation into retained memory. The memory it retains is the resource this entire project is about allocating.

2.5 How big is the cache, concretely

The numbers matter, because they are why paging is worth building. For `gpt2` at this project's default settings (`engine/config.py`: `num_blocks=512`, `block_size=16`), the cache holds $512 \times 16 =$

8192 token slots. GPT-2 has 12 layers, 12 attention heads, and a head dimension of 64. Storing keys *and* values in `float32`:

$$8192 \text{ slots} \times 12 \text{ heads} \times 64 \text{ dims} \times 4 \text{ bytes} \times 12 \text{ layers} \times 2\ (K, V) \approx 604 \text{ MB}$$

The cache is bigger than the model

GPT-2 has about 124 million parameters, roughly 500 MB in `float32`. Its KV cache at this configuration is about 604 MB. The weights are a fixed cost paid once; the cache is a per-user cost that grows with every token every user generates. On a real deployment, the KV cache, not the model, is what runs out first. That is why an entire project exists to manage it.

Note the configuration is deliberately balanced: `max_model_len` is 1024 (GPT-2's architectural limit) and `max_batch_size` is 8. Eight sequences at full length is $8 \times 1024 = 8192$ tokens, exactly the cache capacity. The defaults are sized so that the worst case fits, which is why preemption (section 8.4) has to be provoked deliberately in tests. It is also, as section 13 shows, the only thing standing between the speculative path and a crash.

3 The problem: why naive KV caching wastes most of the memory

The obvious way to store a KV cache is to give each sequence one contiguous block of memory big enough for the longest answer it could produce. This is what a straightforward implementation does, and it is bad in three separate ways at once.

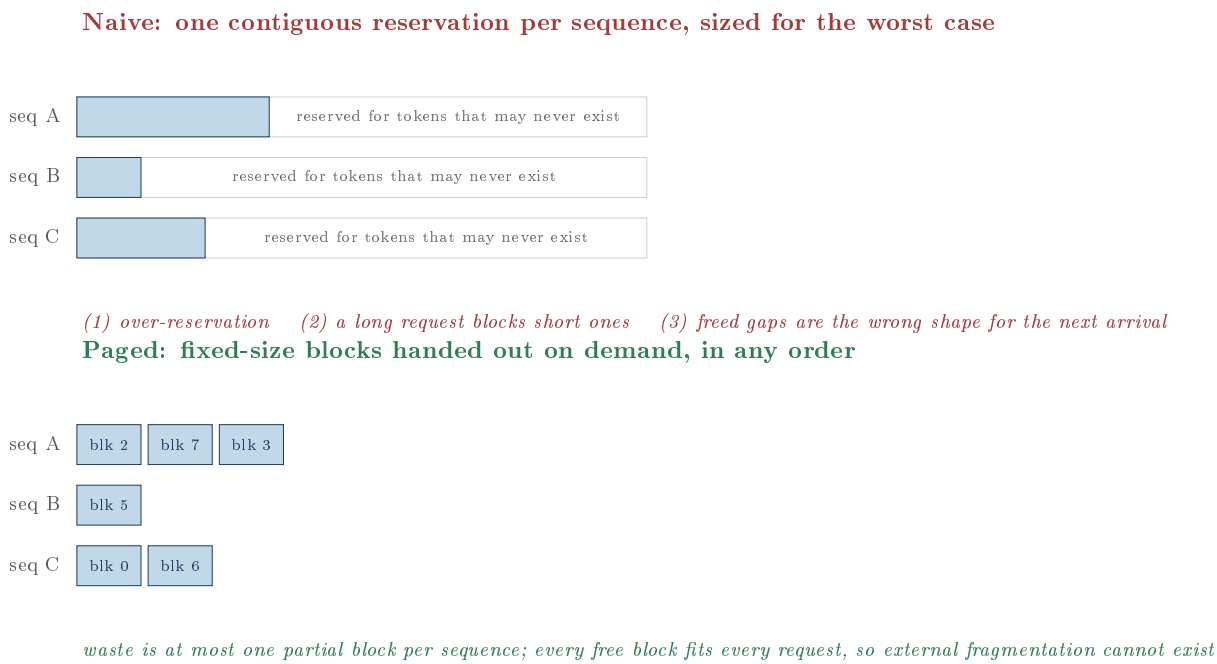


Figure 3: The core reframing the project borrows from operating systems. A sequence stops owning a *region* and starts owning a *list of block numbers*. Physical order stops mattering.

Jargon: Fragmentation, internal and external

External fragmentation is having enough free memory in total but not in one contiguous piece, so a request fails anyway. **Internal** fragmentation is space wasted inside an allocation you did take, because it was rounded up. Fixed-size blocks trade the first for the second: external fragmentation becomes *impossible* (every free block is interchangeable), and internal fragmentation is capped at one block per sequence. That is an excellent trade, and it is

exactly the trade an operating system's virtual memory makes.

Paging, in one paragraph

Your computer already does this. A program thinks it has one long stretch of memory; the operating system actually gives it scattered fixed-size *pages* and keeps a *page table* translating "the address the program thinks it is using" into "where it really is". This project applies that idea one level up: a sequence thinks it has a smooth run of token positions, the engine gives it scattered fixed-size blocks, and a *block table* does the translation. The paper this comes from (vLLM, Kwon et al. 2023) calls the result *PagedAttention*. Everything valuable follows from the translation layer: once addresses are indirect, two sequences can point at the same physical block, and sharing becomes free.

4 Repository map

```
inference-engine
├── engine/                 the whole engine, no framework
│   ├── config.py           EngineConfig, SamplingParams
│   ├── sequence.py         one generation stream's state
│   ├── cache.py            physical KV tensors
│   ├── block_manager.py    allocator: free lists, prefix cache, COW
│   ├── scheduler.py        admission, batching, preemption
│   ├── model.py            GPT-2 forward + paged attention
│   ├── sampling.py         greedy, temperature, top-k, top-p
│   ├── speculative.py      accept/reject sampling
│   ├── llm_engine.py       the synchronous core: step()
│   └── async_engine.py     the step loop in a worker thread
├── server/
│   ├── app.py              FastAPI, OpenAI-compatible, SSE
│   └── __main__.py         python -m server
├── tests/                  7 modules, 55 tests
├── benchmarks/
│   ├── bench.py            sweep / baseline / speculative
│   └── results/            committed JSON with machine info
└── docs/ARCHITECTURE.md
```

Figure 4: The layout. The dependency direction is strictly downward: `server` imports `engine`, `engine` imports nothing of `server`, and `block_manager.py` imports no tensor library at all.

The single best structural decision in this codebase

`engine/block_manager.py` does not import `torch`. It is pure bookkeeping: it hands out integers and returns *instructions* of the form "copy physical block 4 to physical block 9". `engine/cache.py` is the only file that touches KV tensors, and it does what it is told.
This split is why the allocator is testable. `tests/test_block_manager.py` has 15 tests covering fragmentation, refcounting, copy-on-write, LRU eviction and rollback, and it runs in under a second because it never loads a model. Most of the genuinely hard logic in a serving engine is in that file, and the split makes it verifiable in isolation. This is the reusable lesson from the project.

5 Architecture

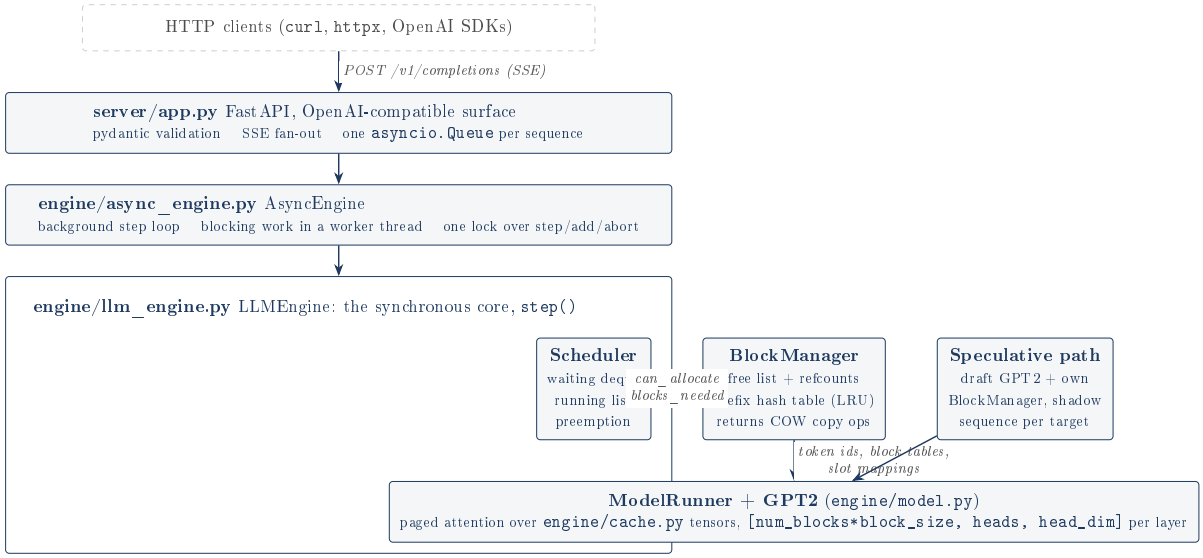


Figure 5: Four layers. Requests flow down, tokens flow back up. Each layer only knows about the one below it.

The reason for four layers rather than one is that they have genuinely different concerns: `app.py` knows about HTTP and knows nothing about blocks; `async_engine.py` knows about threads and knows nothing about attention; `llm_engine.py` knows about tokens and never touches a socket. The sharpest evidence that the split is real is that `llm_engine.py` exposes a plain synchronous `generate()` method used by tests and benchmarks with no server running at all.

6 The paged KV cache

This is the heart of the project. It is split across two files that are deliberately unaware of each other's concerns.

6.1 The physical store: `engine/cache.py`

Only 38 lines, and one idea.

```
class KVCache:
    def __init__(self, num_layers, num_blocks, block_size, num_heads, head_dim,
                 dtype=torch.float32):
        self.block_size = block_size
        shape = (num_blocks * block_size, num_heads, head_dim)
        self.k = [torch.zeros(shape, dtype=dtype) for _ in range(num_layers)]
        self.v = [torch.zeros(shape, dtype=dtype) for _ in range(num_layers)]
```

Listing 1: engine/cache.py, the layout decision

The tensor is *flat*. It is not shaped `[num_blocks, block_size, heads, dim]`; the first two dimensions are multiplied together. That makes a single integer, a *slot id*, address exactly one token's keys and values in one layer:

```
slot = block_id * block_size + offset
```

Why flatten

Because a flat index turns "gather the KV for these 37 scattered token positions" into a single tensor indexing operation, `self.k[layer][slots]`, with no reshaping and no loop. The alternative, a 4-D tensor, would need two-dimensional indexing at every read. This one decision is what lets the attention code stay short.

The class exposes exactly three operations, and this is the whole physical interface to the cache:

Method	Called by	What it does
`write(layer, slots, k, v)`	`Attention.forward`	`index_copy_` new tokens' KV into slots
`gather(layer, slots)`	`Attention.forward`	read KV rows back for attention
`copy_blocks(copies)`	`ModelRunner.apply_copies`	perform copy-on-write duplications

6.2 The bookkeeping: `engine/block_manager.py`

251 lines and no tensors. It tracks, for every physical block:

```python
@dataclass
class Block:
    block_id: int
    ref_count: int = 0
    # Hash over (prefix chain, token ids) once the block is full, else None.
    content_hash: bytes | None = None
```

Listing 2: The entire per-block state

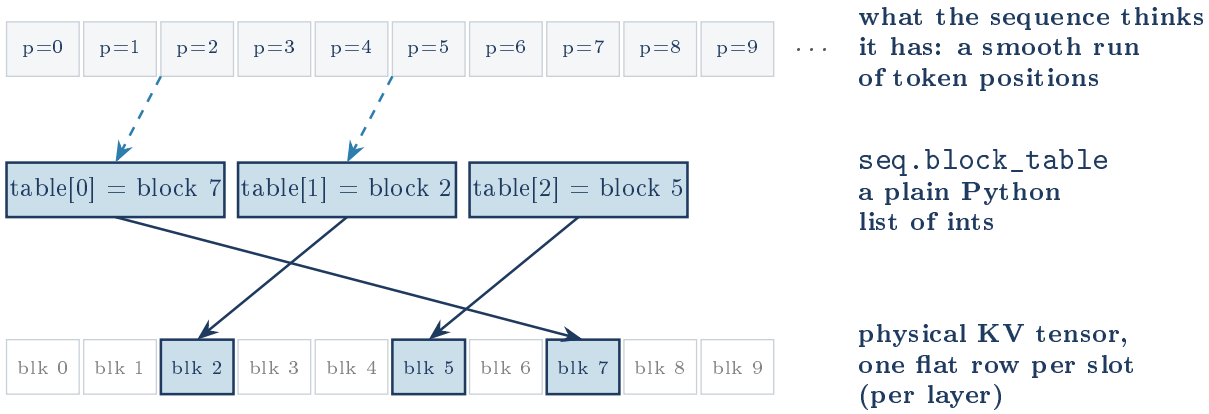
and, for the manager as a whole, four collections:

Field	Type	Meaning
`free_ids`	`deque[int]`	blocks holding nothing anyone wants
`evictable`	`OrderedDict[int, None]`	refcount 0 *but still cached content*, LRU order
`hash_to_block`	`dict[bytes, int]`	content hash to the block holding it
`blocks`	`list[Block]`	per-block refcount and hash

The two-tier free list is the prefix cache

A block with refcount 0 is not necessarily garbage. If it still holds the KV for a recognizable run of tokens, a future request with the same prompt prefix could reuse it *without recomputing those positions at all*. So freeing a hashed block does not return it to `free_ids`; it goes into `evictable`, an LRU queue of blocks that are available but worth keeping.
`num_free_blocks` counts both tiers, because an evictable block is genuinely available: `_pop_free_block` drains `free_ids` first and only cannibalizes the least recently used evictable block when it must. The prefix cache therefore costs nothing in capacity. It is pure upside that occupies memory nobody else has asked for yet.

6.3 Address translation, the centerpiece



The translation, `block_manager.py`, `slot_mapping`:
`slot = block_table[p // block_size] * block_size + p % block_size`
with `block_size = 4` here: position 6 lives in `table[1] = block 2`, offset 2, so its flat slot is `2*4 + 2 = 10`. Physical order is arbitrary and irrelevant.

Figure 6: Logical position to physical slot. This indirection is the entire mechanism; refcounting, prefix caching and copy-on-write are all consequences of it existing.

The implementation is a one-line list comprehension, which is the point:

```
def slot_mapping(self, seq, start, end):
    """Flat physical slot indices for token positions [start, end)."""
    return [seq.block_table[p // self.block_size] * self.block_size
            + p % self.block_size
            for p in range(start, end)]
```

Listing 3: The translation, in full

This is a Python loop on the hot path

`slot_mapping` is called for every sequence on every step, and `ModelRunner.execute` calls it twice per sequence: once for the write slots and once to gather *the entire context* (`slot_mapping(s, 0, lens[b])`). For a sequence with 500 tokens in a batch of 8, that is 4,000 Python-level modulo and division operations per step, before any tensor math happens, and then a `torch.tensor(...)` constructed from that Python list.
On CPU with GPT-2 this is invisible because the model forward dominates by orders of magnitude. On a GPU with a fast model it would be a serious bottleneck, and it is the kind of overhead real engines eliminate by keeping block tables resident on the device as tensors. The project's README is candid about the attention kernel not being truly paged; it does not mention this adjacent cost. Worth knowing before an interviewer asks what would break first on a GPU.

6.4 Allocation: `can_allocate` and `allocate`

Admission is two-phase, and the split is deliberate. `can_allocate` asks "would this fit?" without side effects, so the scheduler can decide without committing. `allocate` then does it, and re-checks:

```
def can_allocate(self, seq):
    matched = self._match_prefix(seq.token_ids)
    matched_evictable = sum(1 for bid in matched if bid in self.evictable)
    needed = seq.num_blocks_needed(self.block_size) - len(matched)
```

```
5       return self.num_free_blocks - matched_evictable >= needed
```

Listing 4: block_manager.py, allocation

The `matched_evictable` term repays a subtle debt. `num_free_blocks` counts evictable blocks as free. But blocks this sequence is about to *reuse* from the prefix cache are also counted there, and it is about to take them out of that pool. Counting them as both "available for the new allocation" and "available to satisfy the remaining need" would double count. Subtracting them fixes it. This is the kind of off-by-a-pool error that produces an empty free list mid-step, and it is handled correctly.

`_match_prefix` runs three times per admission

`Scheduler.schedule` calls `can_allocate`, which calls `_match_prefix`. It then calls `allocate`, which calls `can_allocate` again (which calls `_match_prefix` again), and then calls `_match_prefix` a third time itself. Each call walks the whole prompt computing a chain of SHA-256 hashes, one per full block.
It is correct, and on CPU it is lost in the noise next to a transformer forward pass. It is still three times the necessary work on the admission path, and the result of the first call could simply have been threaded through. A reviewer will spot it; better to have the answer ready ("correct, redundant, and the profile says it does not matter here") than to be surprised by it.

6.5 Refcounting, forks and copy-on-write

Jargon: Reference counting

Each block records how many sequences currently point at it. Sharing is a `+1`. Releasing is a `-1`. At zero, the block is reclaimable. This is the same technique CPython uses to know when your objects can be freed.

Jargon: Copy-on-write (COW)

Two sequences may safely share a block as long as both only *read* it. The instant one wants to *write*, it must first take a private copy, so the other one's data is not corrupted. Deferring the copy until a write actually happens means the copy usually never happens at all. This is exactly what the Unix `fork(2)` system call does with process memory, and the project uses the same name for the same reason.

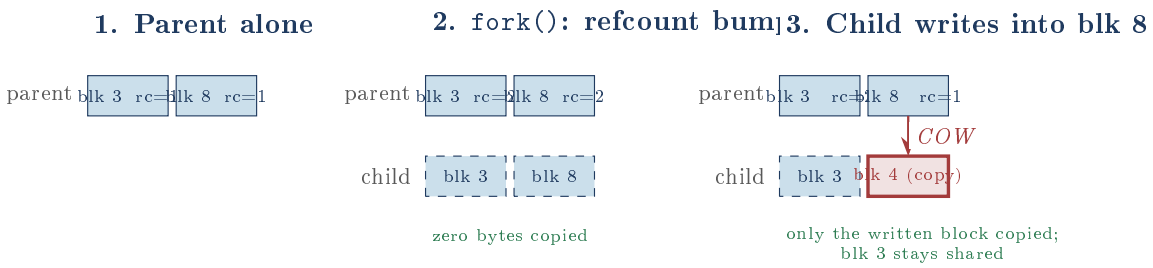


Figure 7: Copy-on-write, as pinned by `test_fork_shares_then_copies`. Note stage 3: the *full* block 3 stays shared forever, because a full block's contents can never change. Only the partially filled tail block is ever copied. Sharing an `n`-token prompt across `n` completions costs one block copy, not `n` prompt recomputations.

The write path is `append_slots`, and it does three jobs in one pass:

```python
def append_slots(self, seq):
    copies = []
    first_write_block = seq.num_computed // self.block_size
    for i in range(first_write_block, len(seq.block_table)):
        bid = seq.block_table[i]
        if self.blocks[bid].ref_count > 1:
            new_bid = self._pop_free_block()      # (1) copy-on-write
            copies.append((bid, new_bid))
            seq.block_table[i] = new_bid
            self._unref_block(bid)
            self.stats.cow_copies += 1
        elif self.blocks[bid].content_hash is not None:
            self._deregister(bid)                 # (2) in-place rewrite
    total = seq.num_blocks_needed(self.block_size)
    while len(seq.block_table) < total:
        seq.block_table.append(self._pop_free_block())   # (3) growth
    ...
    return copies
```

Listing 5: block_manager.py, append_slots (abridged)

Branch (2) is the subtle one and the project calls it out as its hardest bug.

The stale-hash bug, and why it is the interesting one

The prefix cache says "block 8 contains exactly this run of tokens; anyone whose prompt starts that way may reuse it". That promise is only true while block 8's contents match its hash.

Almost nothing rewrites already-computed positions, so the promise almost always holds. But two features do rewrite them: a **speculative rollback** throws away rejected tokens and writes different ones over the same positions, and **stop-string truncation** does the same. If block 8 was hashed and then rewritten in place, the hash table now hands out a block whose contents silently changed, and a future request gets another request's KV data for part of its prompt. The output would be subtly, unreproducibly wrong.

The fix is one branch: the instant a registered block is about to be written in place, drop its hash. `test_truncate_deregisters_rewritten_block` pins it. This is the kind of bug that only exists once two independently correct features (prefix caching, speculative decoding) meet, which is exactly what makes it worth the story.

6.6 Prefix caching and the hash chain

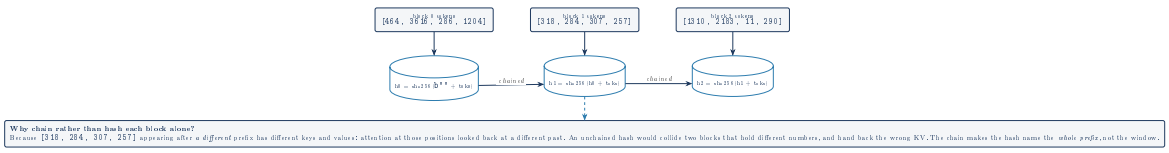


Figure 8: The prefix hash chain, `_hash_block` in `block_manager.py`. Only *full* blocks are hashed: a partial block's contents are still changing, so it has no stable identity.

The prefix cache trusts SHA-256 to not collide, silently

`hash_to_block` maps a 32-byte digest to a block id and never verifies that the block's tokens actually match. A collision would serve one request's KV cache to another. This is the correct engineering call, a SHA-256 collision is not a thing that happens, and vLLM makes the same call, but it is worth being able to say out loud that the cache's correctness rests on a cryptographic assumption rather than a comparison. Storing the token ids alongside and

checking on hit would cost one list comparison per hit and remove the assumption entirely.

There is one non-obvious rule in `_match_prefix` worth pulling out:

```
max_match_tokens = len(token_ids) - 1
num_full = max_match_tokens // self.block_size
```

Why a fully cached prompt still recomputes its last block

Suppose a request's prompt is *identical* to one served a moment ago, so every block of it is in the prefix cache. If the engine matched all of it, it would have nothing to run the model on, and the model produces logits only for positions it actually computes. With zero positions to compute, there are no logits, and no first token.
So matching is capped at `len(token_ids) - 1` tokens, guaranteeing at least one position is always recomputed. `test_no_match_capped_below_full_prompt` pins it: an 8-token prompt with all 8 tokens cached reports `num_cached_tokens == 4`, not 8. It is a one-line rule that would produce a baffling crash if it were missing.

6.7 Capacity accounting: `blocks_needed`

```
def blocks_needed(self, seq, num_new_tokens):
    new_len = len(seq.token_ids) + num_new_tokens
    total = (new_len + self.block_size - 1) // self.block_size
    growth = max(0, total - len(seq.block_table))
    first_write_block = seq.num_computed // self.block_size
    cow = sum(1 for i in range(first_write_block, len(seq.block_table))
              if self.blocks[seq.block_table[i]].ref_count > 1)
    return growth + cow
```

Listing 6: The worst-case demand estimate

Two demands are summed: blocks needed because the sequence got *longer*, and blocks needed because it is about to write into blocks it *shares*. Forgetting the second term would let the engine admit a batch that fits by length and then run out of blocks halfway through resolving copy-on-write. `test_can_append_accounts_for_cow` pins exactly that.

The invariant that makes `num_new_tokens=0` correct

`Scheduler.schedule` calls `blocks_needed(s, self.lookahead)` where `lookahead` is 0 without speculative decoding. Asking "how many blocks to add zero tokens?" looks like a bug, and is not.
`sequence.py` documents the invariant: between steps, `num_computed == len(token_ids) - 1`. The token sampled at the end of the previous step is *already appended* to `token_ids`; it simply has not been fed through the model yet. So the sequence's length already accounts for the token about to be computed, and zero further tokens need to be added. With speculative decoding, `lookahead = k` because the draft is about to append `k` more.
This is genuinely elegant and genuinely confusing. Anyone reading `scheduler.py` in isolation will read it as an off-by-one. The invariant is documented in `sequence.py` and nowhere near the call site.

7 The model: GPT-2 rebuilt, with paged attention

`engine/model.py` is an independent implementation of the GPT-2 forward pass. Hugging Face is asked for the trained numbers and nothing else.

7.1 Loading weights, and the transpose that matters

```
# HF Conv1D stores weights as [in, out]; nn.Linear wants [out, in].
for name in ("attn.c_attn", "attn.c_proj", "mlp.c_fc", "mlp.c_proj"):
    new_sd[f"{dst}.{name}.weight"] = sd[f"{src}.{name}.weight"].t().contiguous()
    new_sd[f"{dst}.{name}.bias"] = sd[f"{src}.{name}.bias"]
```

Listing 7: model.py, from_hf

The original GPT-2 used a layer called `Conv1D` which stores its matrix transposed relative to PyTorch's standard `nn.Linear`. Loading the weights without the `.t()` produces a model that runs, has no error, and emits nonsense. This class of bug is why the parity test exists.

Verify against a reference before building anything on top

`tests/test_parity.py` runs the same prompts through this implementation and through Hugging Face's, and asserts the largest disagreement in any of the 50,257 logits is under `1e-3`. The README reports the measured values: about `3e-5` on distilgpt2 and `2e-4` on gpt2, which is float32 rounding noise, not a difference in the math.
The payoff is stated in the README's own "what I learned" and it is the right lesson: with parity established, every later bug is a *systems* bug. When the paged attention path produced wrong output, the question was never "did I get the transformer wrong?". A second test, `test_paged_forward_matches_cache_free`, extends the same discipline to the cache: the paged path and the plain causal path must agree to `1e-4`.

7.2 Attention against a paged cache

The attention module has two modes, chosen by whether a `ForwardContext` is supplied. Without one, it is a textbook causal attention used by the parity tests. With one, it is the paged path:

```
ctx.cache.write(layer_idx, ctx.slot_mapping,
                k.reshape(B * T, self.n_head, self.head_dim),
                v.reshape(B * T, self.n_head, self.head_dim))
keys, values = ctx.cache.gather(layer_idx, ctx.gather_slots.reshape(-1))
Lmax = ctx.gather_slots.shape[1]
keys = keys.view(B, Lmax, self.n_head, self.head_dim).transpose(1, 2)
values = values.view(B, Lmax, self.n_head, self.head_dim).transpose(1, 2)
out = F.scaled_dot_product_attention(q.transpose(1, 2), keys, values,
                                     attn_mask=ctx.attn_mask)
```

Listing 8: model.py, the paged attention path

The ordering is load-bearing: **write before gather**. The new token's own key and value must be in the cache before the gather reads the context, because a token attends to itself. Reversing those two lines would silently drop the diagonal of the attention matrix.

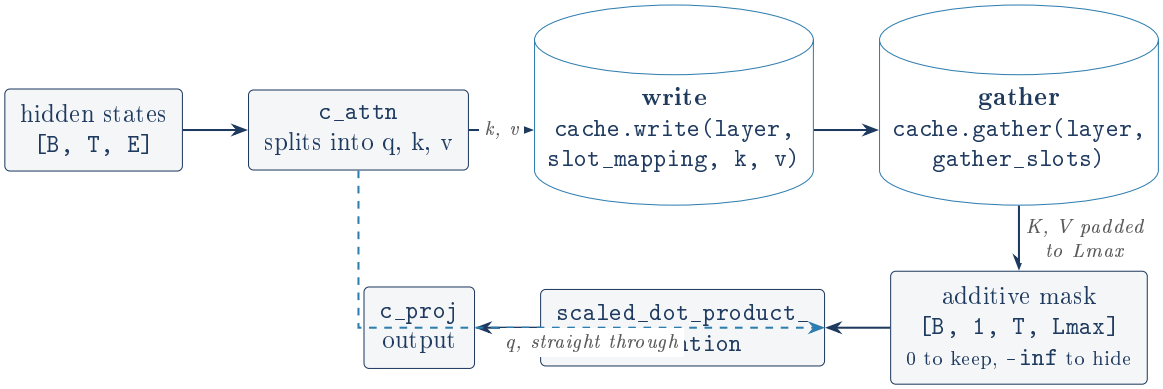


Figure 9: One attention layer's paged path. Every layer shares the same `ForwardContext`, so the slot arithmetic is done once per forward, not twelve times.

The mask construction deserves a look because it handles prefill, decode and speculative verification with the same three lines:

```
key_pos = torch.arange(Lmax).view(1, 1, Lmax)
limit = torch.tensor([s.num_computed for s in seqs]).view(B, 1, 1) \
    + torch.arange(T).view(1, T, 1)
mask = torch.where(key_pos <= limit, 0.0, NEG_INF).view(B, 1, T, Lmax)
```

Listing 9: model.py, one mask for every mode

Query row `t` of sequence `b` may attend to key positions up to `num_computed[b] + t`. When `T = 1` (decode) this keeps exactly the sequence's real length and hides the padding. When `T = k+1` (speculative verification) it reconstructs a causal triangle across the drafted tokens. Sequences shorter than `Lmax` have their padding slots pointing at physical slot 0, which holds some other sequence's real data, and the mask is what makes that harmless: those columns are always beyond the limit and get `-inf`.

The attention is not actually paged, and the README says so

This is the project's own headline caveat and the code confirms it exactly. `gather` pulls each sequence's scattered KV into a *dense padded* tensor of shape `[B, Lmax, H, D]` and hands it to a standard attention kernel. So:

- The **memory** savings of paging are completely real. Nothing is over-reserved and nothing is fragmented.
- The **compute** savings are not. A batch of one 500-token sequence and seven 20-token sequences pads all eight to 500 and does attention arithmetic on 3,360 positions of nothing.
- The gather itself *materializes a copy* of the whole batch's context, every layer, every step. That allocation is proportional to `B * Lmax * H * D` and is the thing paging was supposed to avoid touching.

A real engine writes a fused kernel that walks the block table inside the attention computation. The README states this plainly and gives the honest reason for not doing it (on CPU with a small model it was not the bottleneck). That is a defensible call and a well-documented one. It is also the single biggest gap between this project and the systems it studies, so it should be the first thing volunteered, not the first thing discovered.

8 Sequences, scheduling and continuous batching

8.1 What a sequence is

`engine/sequence.py` is a dataclass and its docstring carries the invariant everything else depends on:

```python
@dataclass
class Sequence:
    seq_id: int
    token_ids: list[int]        # prompt + generated, always
    params: SamplingParams
    arrival_time: float = 0.0
    status: SequenceStatus = SequenceStatus.WAITING
    block_table: list[int] = field(default_factory=list)
    num_prompt_tokens: int = 0
    num_computed: int = 0       # how many have KV in the cache
    num_cached_tokens: int = 0  # how many came free from the prefix cache
    finish_reason: FinishReason | None = None
    generator: torch.Generator | None = None
    num_preemptions: int = 0
```

Listing 10: sequence.py, the state

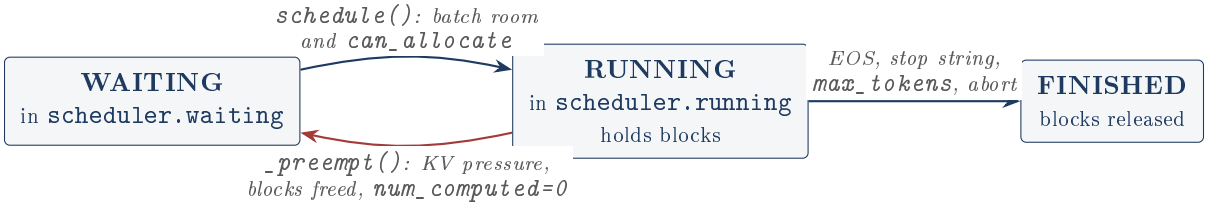


Figure 10: The sequence state machine. Preemption is the only backward edge, and it is total: the sequence loses its blocks and its progress markers, keeping only its `token_ids`.

8.2 Static versus continuous batching

Jargon: Batching

Running several sequences through the model together in one call. It is faster than running them one after another because the model's weights get loaded from memory once and reused across the whole batch, and because matrix multiplication hardware is far more efficient on a big matrix than on several small ones.

The naive way to batch is to collect a group of requests, run them together, and return when they are all done. The problem is that they do not finish together.

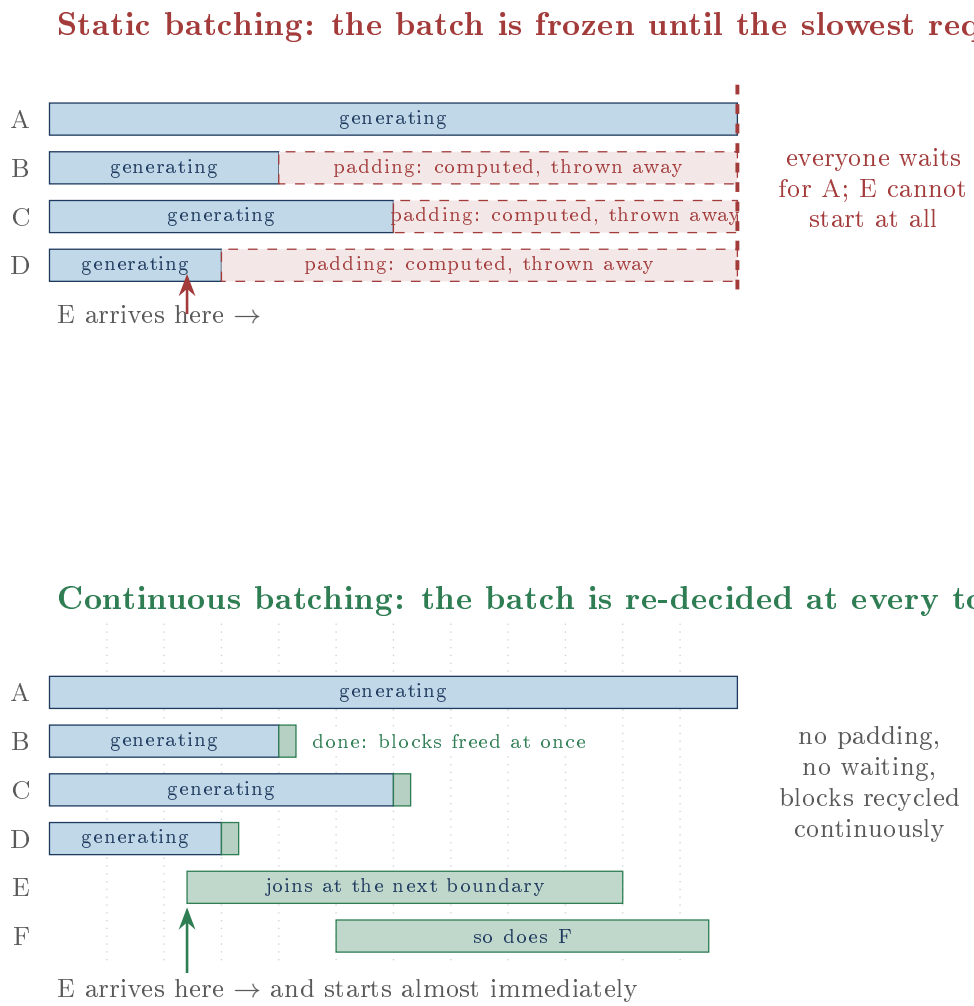


Figure 11: The mechanism behind the project's measured 2.5x throughput gain. The win is not from making any single request faster; it is from never computing padding and never idling. The dotted lines are token boundaries: the only points at which the scheduler is allowed to change its mind.

8.3 The scheduler

`engine/scheduler.py` is 95 lines and runs once per step. It does two things in order.

First, admit. FIFO, while there is both batch room and KV space:

```
while self.waiting and len(self.running) < self.config.max_batch_size:
    seq = self.waiting[0]
    if not self.block_manager.can_allocate(seq):
        break
    self.waiting.popleft()
    self.block_manager.allocate(seq)
    seq.status = SequenceStatus.RUNNING
    self.running.append(seq)
    out.prefill.append(seq)
```

Note the `break`, not `continue`. A prompt too big to fit blocks everything behind it rather than being skipped over. That is strict FIFO with no starvation, at the cost of head-of-line blocking. It is a real trade and it is made silently.

Second, guarantee the step can happen. This is the interesting half:

```
decode = [s for s in self.running if s not in out.prefill]
while decode:
    needed = sum(self.block_manager.blocks_needed(s, self.lookahead)
                 for s in decode)
```

```
    if needed <= self.block_manager.num_free_blocks:
        break
    victim = max(self.running, key=lambda s: (s.arrival_time, s.seq_id))
    self._preempt(victim)
    out.preempted.append(victim)
    ...
```

Listing 11: scheduler.py, the cumulative check

Cumulative, not per-sequence: the bug worth telling

The obvious implementation checks each sequence against the free list on its own. With three sequences each needing one block and two blocks free, every individual check passes ("I need 1, there are 2 free"), the batch is admitted, and the third sequence crashes mid-step against an empty free list.

The check must be `sum(...)` across the whole batch, because the sequences are not competing with the free list, they are competing with *each other*. The project's README names this as its second-hardest bug and credits `test_scheduler_invariants_under_churn` with catching it. That test is worth reading: it runs 300 scheduling steps over six sequences in a cache deliberately too small, and asserts after every single one that `used + num_free_blocks == num_blocks`. It is a hand-rolled property test, and it is what turns "I think the accounting is right" into evidence.

8.4 Preemption

Jargon: Preemption

Kicking a running job out to make room, intending to resume it later. Your operating system preempts processes off the CPU constantly. Here the scarce resource is not CPU but KV cache blocks.

```
def _preempt(self, seq):
    self.block_manager.free_seq(seq)
    seq.num_computed = 0
    seq.num_cached_tokens = 0
    seq.num_preemptions += 1
    seq.status = SequenceStatus.WAITING
    self.running.remove(seq)
    self.waiting.appendleft(seq)      # front, not back
    self.num_preemptions += 1
```

Listing 12: scheduler.py, _preempt

Three decisions are packed into nine lines:

1. **Victim selection: newest arrival.** `max(...)` by `(arrival_time, seq_id)`. Sacrificing the newest protects the oldest, which has the most computed tokens to lose and has waited longest. Evicting the oldest instead would risk starving it forever.
2. **Requeue at the *front*.** `appendleft`, so the victim is first in line when space appears. Without this, a preempted sequence could be starved behind arrivals that were not even submitted when it started.
3. **Recompute, do not swap.** The blocks are gone; the sequence keeps only `token_ids` and redoes its prefill from scratch. The alternative, copying KV out to slower memory and back, is what a GPU engine does. On CPU the "device" memory and host memory are the same memory, so swapping would be a pure-overhead no-op. The README makes this argument and it is correct.

Why recompute is cheaper than it sounds

Preemption throws away the prompt's KV, and re-admission has to prefill again. But the prefix cache is still holding those exact blocks in its evictable pool. Unless they were cannibalized in the meantime, re-admission gets most of the prompt back for a refcount bump. The two features compose: preemption is cheap *because* prefix caching exists.

A just-admitted prompt can be preempted in the same step

`victim = max(self.running, ...)` scans everything running, including sequences admitted moments earlier in the same `schedule()` call. Because the victim is the newest arrival, and a just-admitted sequence usually *is* the newest arrival, the loop can allocate a prompt's blocks and then immediately free them again. The code handles it correctly (`out.prefill.remove(victim)` keeps the decision consistent), so this is wasted work, not a bug: an `allocate` and a `free_seq` for nothing, plus the sequence goes back to the front of the queue it just left. Admitting only after confirming the decode batch fits would avoid the churn.

9 The engine step

`engine/llm_engine.py` is the largest file (357 lines) and `step()` is the function the whole project exists to make possible.

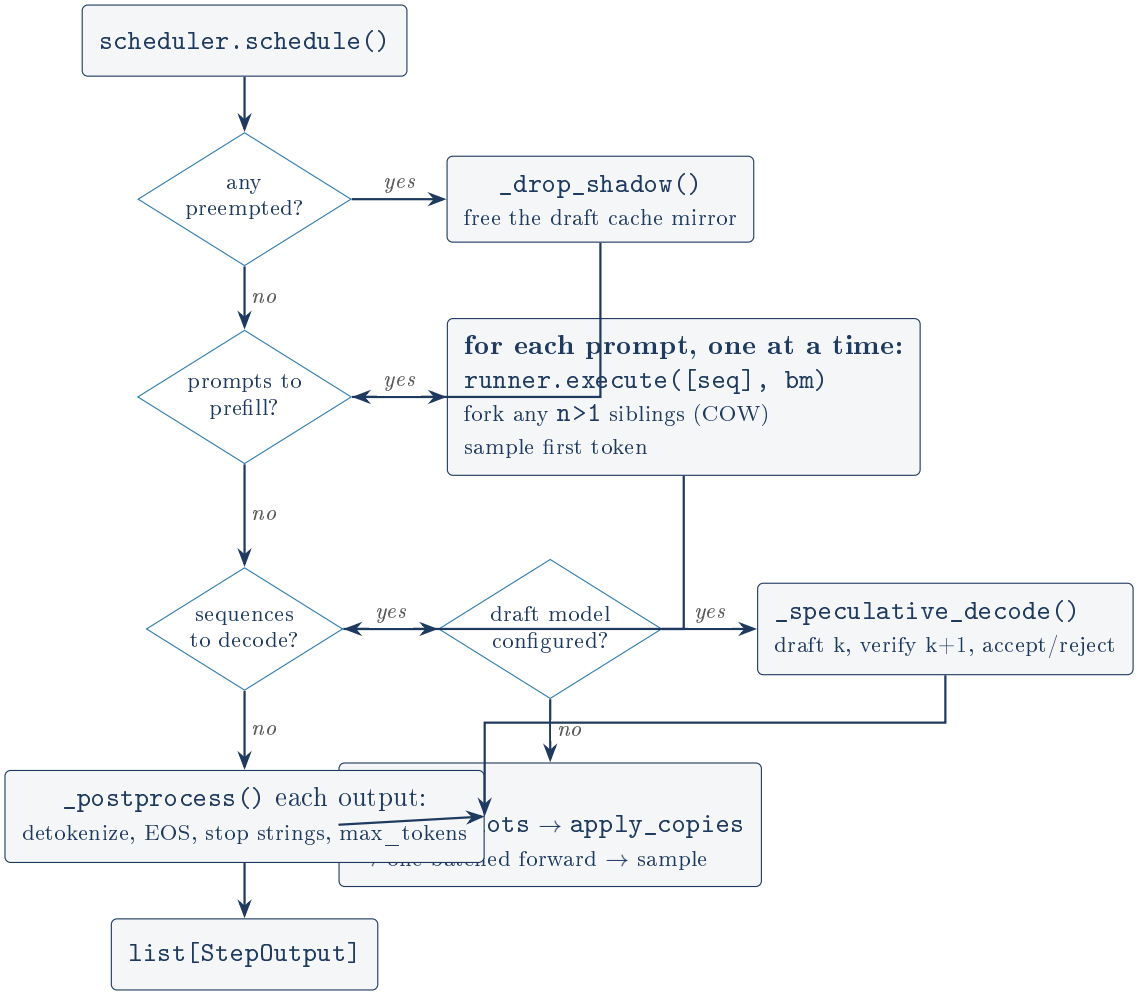


Figure 12: One `step()`. Every running sequence advances by exactly one token boundary: one token normally, up to `k+1` with speculative decoding.

9.1 Prefill and decode are different shapes

Jargon: Prefill and decode

Prefill is the first pass over a new prompt: many token positions go in at once, and only the last one's logits are wanted. It is compute-heavy. **Decode** is every subsequent step: exactly one token position per sequence goes in. It is memory-bandwidth-heavy, because the model's whole weight matrix has to be read to produce a single token. This asymmetry is why batching helps decode so much: eight sequences decoding together read the weights once instead of eight times.

The engine runs them differently, and `ModelRunner.execute` enforces why:

```
T = len(seqs[0]) - seqs[0].num_computed
assert T >= 1 and all(len(s) - s.num_computed == T for s in seqs)
```

Every sequence in one call must contribute the *same* number of new positions, because the input is a rectangular tensor `[B, T]`. Decode satisfies this trivially (`T=1` for everyone). Speculative verification satisfies it because `k` is chosen as a minimum over the batch. Prefill cannot: prompts have different lengths. Hence:

```
for seq in decision.prefill:
    logits = self.runner.execute([seq], self.block_manager)
```

Listing 13: llm_engine.py, step(): prefills are unbatched

Unbatched prefill is what the TTFT numbers are actually measuring

The README lists "prefills run one at a time" under "what I'd do differently" and gives a reason (CPU prefill is already compute-bound). But it does not connect it to its own benchmark table, and the connection is direct.

Time-to-first-token in `benchmarks/results/serving_sweep.json` goes 0.069s, 0.266s, 0.541s, 1.266s at concurrency 1, 4, 8, 16. That is very close to linear in concurrency, and a single sequential prefill of these prompts takes about 0.07s. Eight simultaneous arrivals means eight prefills executed one after another, and the eighth client waits for all of them: $8 \times 0.07 \approx 0.55$s, against a measured 0.541s.

So the TTFT curve is not a general property of continuous batching. It is a direct readout of this specific unbatched-prefill decision. The README describes the TTFT growth as "the expected tradeoff", which understates it: batching ragged prompts (as the README itself proposes) would flatten most of that curve. This is a strong point to make voluntarily, because it demonstrates reading one's own benchmark rather than reporting it. *(This paragraph's arithmetic is an inference drawn from the committed JSON and the code, not a separately measured claim.)*

9.2 Forking for `n > 1`

When a request asks for several completions of one prompt, re-running the prompt per completion would be pure waste. Instead siblings are held back until the parent's prefill has produced its KV, then forked onto it:

```
for child in self._pending_forks.pop(seq.seq_id, []):
    tok = sample_token(logits[0, -1], child.params, child.token_ids,
                       child.generator)
    if len(self.scheduler.running) < self.config.max_batch_size:
        self.block_manager.fork(seq, child)
        child.append_token(tok)
        child.status = SequenceStatus.RUNNING
        self.scheduler.running.append(child)
        outputs.append(self._postprocess(child, [tok]))
    else:
        # No batch room: requeue as an ordinary request; the
        # prefix cache makes its prefill nearly free.
        self.scheduler.add(child)
```

Listing 14: llm_engine.py, step(): sibling forks

The siblings share the parent's *entire* prompt (figure 7) and diverge only when one writes, which happens on its first decode step and costs exactly one block copy. They sample different tokens from the same logits because `add_request` gives each sibling `i` its own generator seeded with `params.seed + i`.

Seeded `n > 1` is not reproducible under load, and nothing tests it

Look at the `else` branch. When there is no batch room, the child has *already consumed a random draw* from its generator via `sample_token` on the line above, and that token is then discarded and the child requeued. Its eventual prefill samples again, from a generator that has advanced by one draw.

So the same request, with the same seed, produces different output depending on whether the batch happened to be full at that instant. That makes seeded determinism, an explicitly

advertised feature ("seeded determinism" in the README's feature list), conditional on server load for `n > 1`.
The test suite does not catch it: `test_n_completions_fork_with_cow` uses `max_batch_size=4` with `n=2`, so the branch never fires, and `test_n_gives_multiple_choices` checks that two choices come back but never checks reproducibility. The fix is small (sample the child's token after deciding which path it takes, or do not sample at all on the requeue path). The finding is not that the bug is severe; it is that a documented guarantee has a load-dependent hole with no test on it.

9.3 Detokenization and the UTF-8 holdback

Jargon: UTF-8, and why a character can be split

UTF-8 encodes most characters in one byte but many in two to four. GPT-2's tokenizer works on bytes, so a token boundary can land *in the middle* of a character. Decoding just the first half yields the Unicode replacement character, the black-diamond question mark shown when text is broken.

```
class Detokenizer:
    def delta(self, seq, state):
        text = self.tokenizer.decode(seq.output_token_ids,
                                     skip_special_tokens=True)
        # if text ends with the Unicode replacement character U+FFFD:
        #     return ""      <- hold back, wait for the next token
        delta = text[len(state.emitted_text):]
        state.emitted_text = text
        return delta
```

Listing 15: llm_engine.py, Detokenizer (the guard condition is described below, not shown)

Why the code cannot be pasted verbatim here

The real guard is `if text.endswith(...)` comparing against a literal U+FFFD character. That character is multi-byte, and `pdflatex` has no glyph for it; pasting it into a code listing is a hard build failure, not a cosmetic one. It is described rather than shown, deliberately. The logic is: if the decoded text ends in a broken character, emit nothing this step and try again next step, when the second half of the character will have arrived.

The detokenizer is quadratic in the output length

`delta()` decodes `seq.output_token_ids` *in full*, every step, and then slices off the part already emitted. Generating n tokens therefore performs n decodes of average length $n/2$: $O(n^2)$ work, plus $O(n^2)$ total string allocation, plus `state.emitted_text` holding a fresh copy of the whole output each step.
At the benchmark's 32 tokens this is free. At 1000 tokens (GPT-2's limit) it is half a million token-decodes per request, and on a GPU where the forward pass is fast it could plausibly become a visible fraction of step time. The straightforward fix, which real engines implement, is to keep a running byte buffer and decode only the new tokens, holding back an incomplete tail.

9.4 Finishing: EOS, `max_tokens`, stop strings

`_postprocess` applies three termination conditions in a fixed order, and each may have to *retract* tokens that were already appended, because speculative decoding emits several tokens at once

and only then discovers one of them was the end.

Condition	Detected on	Retraction
EOS token (id 50256)	token ids	`_truncate_tail` from the EOS onward
`max_tokens`	count	`_truncate_tail` the overshoot
stop string	decoded text	trim the *text* only; sequence ends anyway

`_truncate_tail` is the important one, because it must keep the block table honest, not just the token list:

```
def _truncate_tail(self, seq, drop):
    seq.token_ids = seq.token_ids[:len(seq.token_ids) - drop]
    seq.num_computed = min(seq.num_computed, len(seq.token_ids) - 1)
    self.block_manager.truncate(seq, len(seq.token_ids))
```

This is the second caller (with speculative rollback) of the path that made the stale-hash bug possible.

`EOS_TOKEN_ID = 50256` is a hardcoded module constant

The tokenizer knows its own EOS id (`tokenizer.eos_token_id`), and the engine already holds the tokenizer. Hardcoding the GPT-2 value at module level means the engine is silently GPT-2-family-only in a way the configuration does not express: point `--model` at any other architecture and the engine will load it, run it, and never stop generating, because 50256 means something else there. Given the project's stated scope this is defensible, but it is a one-character-of-effort fix that would remove a whole class of confusing failure.

`seq_states` grows forever: a real leak in a long-lived server

`add_request` inserts `self.seq_states[seq.seq_id] = _SeqState()` for every sequence. `abort()` pops the entry for sequences that were still waiting, and for cancelled sibling forks. But `_finish()`, the path taken by every sequence that completes *normally*, never pops it.
So a server accumulates one `_SeqState` per request served, forever, and each one holds `emitted_text`: the complete generated text of that request. A server answering a request a second with 200-character answers retains roughly 17 MB of dead completion text per day, alongside a dict that never stops growing.
Nothing catches it, because every test and benchmark constructs an engine, runs a handful of requests, and exits. This is exactly the class of defect that only appears in the thing the project is named after: a long-running service. It is verified in section 13, not merely read off the source.

10 Sampling

`engine/sampling.py` is 68 lines and turns 50,257 logits into one token id. The order of operations is the whole content.

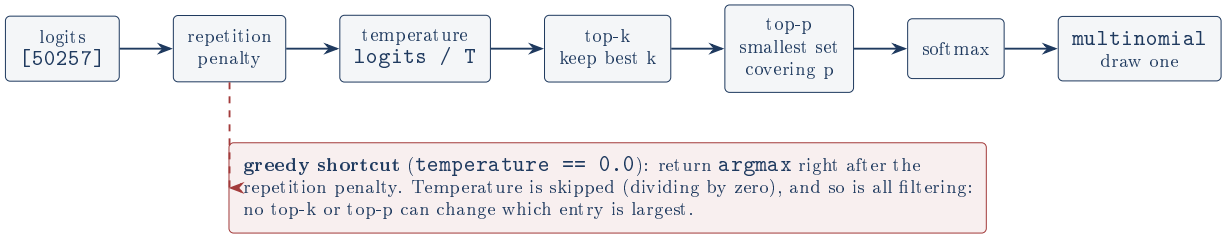


Figure 13: The sampling pipeline, `adjust_logits` then `filter_logits` then draw. Each stage is one small function with its own test in `tests/test_sampling.py`.

Jargon: Temperature, top-k, top-p (nucleus)

Temperature divides the logits before softmax. Below 1 it sharpens the distribution toward the favourite; above 1 it flattens it. At exactly 0 it means "always take the favourite", which is *greedy* decoding. **Top-k** keeps only the k highest-scoring tokens and discards the rest. **Top-p**, or nucleus sampling, keeps the smallest set of tokens whose probabilities add up to p, so the cut adapts: a confident position keeps few candidates, an uncertain one keeps many.

Two details in `sampling.py` are worth stopping on. The repetition penalty handles signs correctly, which a naive implementation gets wrong:

```
logits[seen] = torch.where(
    vals > 0,
    vals / params.repetition_penalty,   # positive logit: divide it down
    vals * params.repetition_penalty)   # negative logit: multiply it down
```

Dividing a *negative* logit by a penalty greater than 1 makes it larger, so "penalizing" a disfavoured token would promote it. The sign split is the standard fix, and both branches are pinned by the test `test_repetition_penalty_discourages_seen_tokens`.

And top-p always keeps at least one token:

```
remove = cum - torch.softmax(sorted_logits, dim=-1) >= params.top_p
remove[0] = False
```

Subtracting each token's own probability from the cumulative sum implements "the token that crosses the threshold stays in". `remove[0] = False` handles the pathological case where the single most likely token already exceeds `top_p` on its own, which would otherwise mask out the entire vocabulary and produce a softmax over nothing but `-inf`. `test_top_p_always_keeps_best_token` pins it.

A greedy request silently ignores `top_k` and `top_p`

`sample_token` returns `argmax` immediately when `params.greedy`, before `filter_logits` runs. This is arithmetically harmless: filtering only ever sets entries to `-inf`, and `top_k >= 1` always keeps the maximum, so the argmax is identical either way. But note that `probs_for`, the function used by the speculative path, does it the other way round: it filters first and *then* builds the one-hot. The two functions that must agree for speculative decoding to be correct reach the same answer by different routes. They do agree, for the reason just given, but the reasoning is load-bearing and written down nowhere.

11 Speculative decoding

This is the most mathematically interesting part of the project, and the one the project is most honest about.

11.1 The idea

Why guessing ahead can be free

Decode is *memory-bound*, not compute-bound. Producing one token requires reading every weight in the model out of memory, and the arithmetic units spend most of that time idle. Crucially, running the model over *five* token positions costs barely more than running it over one: the weights are read once either way.

So: let a small, cheap model guess the next k tokens one at a time. Then run the big model *once* over all k guesses at the same time and ask "would I have said that?" for each. Every guess it agrees with is a token that cost you a cheap forward instead of an expensive one. You get $k+1$ tokens from one big forward if all guesses land, and never fewer than 1 if none do.

The catch, and the reason the technique is subtle, is that this must not change the output distribution. A user must not be able to tell that a smaller model was involved.

Jargon: Draft model and target model

The **target** is the model you actually want answers from (`gpt2` here, 124M parameters). The **draft** is a smaller, faster stand-in that proposes tokens (`distilgpt2`, 82M, a distilled version of GPT-2, so it tends to agree). Only the target's opinion counts. The draft is a guess machine whose guesses are always checked.

11.2 The accept/reject rule

Let $q(x)$ be the draft's probability of the token it proposed and $p(x)$ the target's probability of the same token. The rule (Leviathan et al., 2023) is:

- Accept the draft token with probability $\min(1, p(x)/q(x))$.
- On the first rejection, discard it and every later draft, and sample a replacement from the *residual* distribution, normalized $\max(p-q, 0)$.
- If every token is accepted, take a free bonus token from the target's distribution at position $k+1$, which the verify pass already computed.

Why that exact rule, in words

If the target likes the token at least as much as the draft did ($p \geq q$), keep it unconditionally. If the target likes it *less*, keep it only p/q of the time, which is exactly the fraction that restores the target's rate.

That leaves a deficit: tokens the target wanted more than the draft proposed them. The residual $\max(p-q, 0)$ is precisely the shape of that deficit, and sampling the replacement from it fills the gap exactly. Add the two paths together and the probability of emitting any given token is p again, exactly.

This is the guarantee: the output is distributed *identically* to sampling from the target alone. Not similarly. Identically. Speculative decoding is not an approximation and does not trade quality for speed.

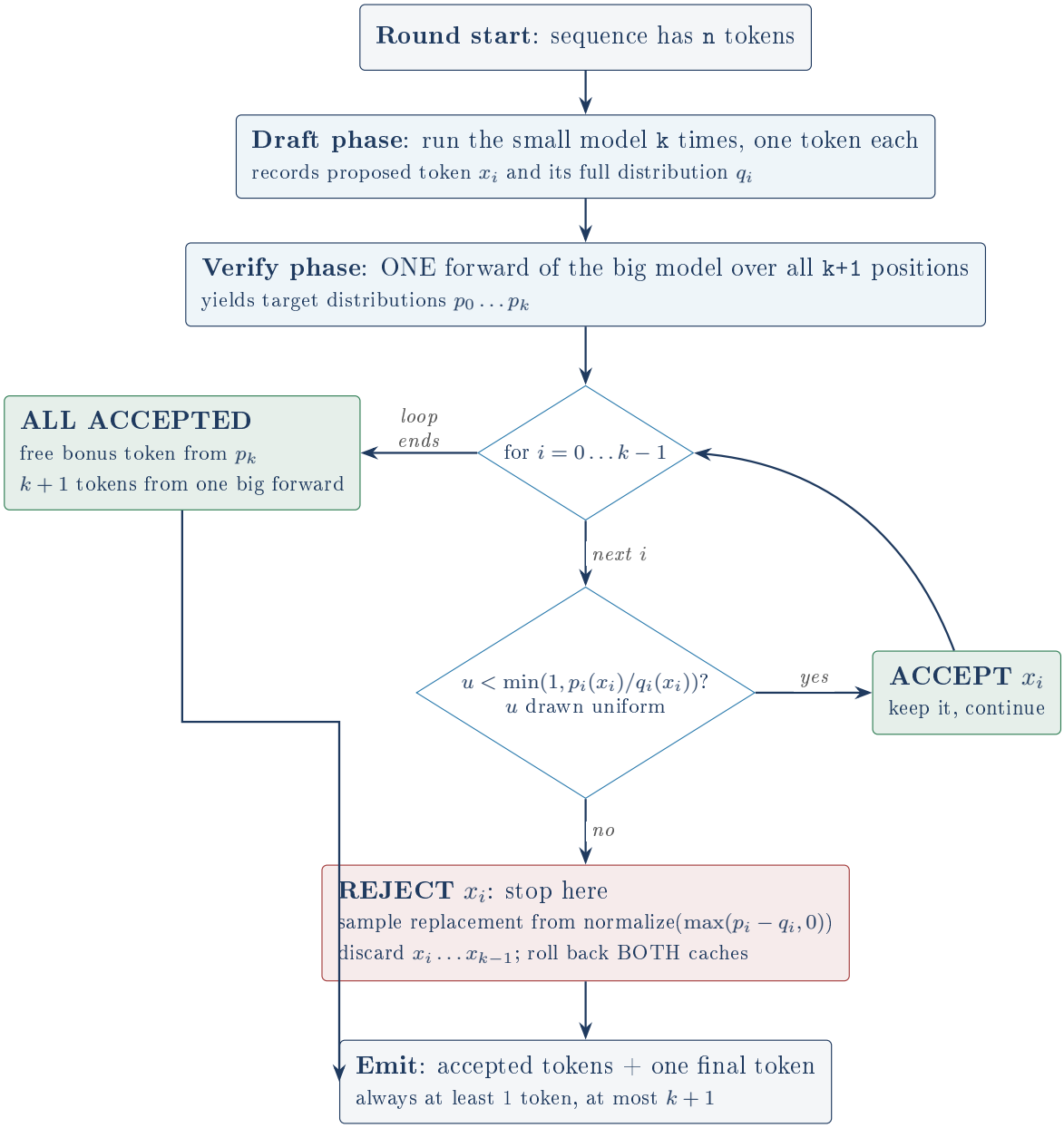


Figure 14: `accept_reject` in `engine/speculative.py`. The important structural property: the draft phase is `k` cheap sequential forwards, the verify phase is **one** expensive forward, and the loop never runs the big model again regardless of what it decides.

11.3 The greedy degeneration

With `temperature = 0`, `probs_for` returns a one-hot vector. Then $q(x) = 1$ for the drafted token, and $p(x)$ is either 1 (the target agrees) or 0 (it does not). The ratio is 1 or 0: accept when the two models' argmaxes match, reject otherwise, and the residual $\max(p - q, 0)$ is the target's own argmax. The general rule collapses to "accept while the draft agrees with the target", with no randomness consumed at all:

```
ratio = 1.0 if q_tok == 0.0 else min(1.0, p_tok / q_tok)
u = float(torch.rand((), generator=generator)) if ratio < 1.0 else 0.0
if u < ratio:
    accepted.append(tok)
    continue
```

Listing 16: speculative.py, accept_reject: no draw when ratio is 1

The `if ratio < 1.0` guard is what makes greedy speculative output *bit-identical* to greedy plain output rather than merely equivalent in distribution: no generator draw is consumed, so nothing downstream shifts. `test_spec_engine_greedy_matches_plain_engine` asserts exactly that, comparing token ids, not text.

11.4 How the correctness claim is actually evidenced

This is the strongest part of the test suite and deserves to be named as such.

- `greedy_accept_reject_is_exact`
  Hand-built one-hot vectors; asserts the exact accept/reject decision. Unit-level.
- `identical_models_accept_everything`
  Draft = target, so $p = q$ everywhere: the acceptance rate must exceed 0.999. Catches a mis-signed ratio.
- `empirical_distribution_matches_target`
  The real proof. Two tiny random transformers, a 13-token vocabulary, 4000 full speculative rounds; asserts the total variation distance between the empirical output distribution and the target's true distribution is under 0.05.
- `empirical_distribution_with_temperature_and_top_k`
  The same, with temperature 0.7 and top-k 6, proving the guarantee survives logit transformation.

(Each name above is prefixed `test_` in `tests/test_speculative.py`.)

Jargon: Total variation distance

A measure of how far apart two probability distributions are: $\frac{1}{2}\sum_x |P(x)-Q(x)|$. Zero means identical; one means they never agree. It is the right metric here because the claim is about distributions, not about individual tokens.

Testing a probabilistic guarantee empirically, correctly

The claim "the output is distributed exactly as the target" cannot be checked by comparing two outputs; every run is random. The project's answer is to shrink the problem until statistics become decisive: a vocabulary of 13 instead of 50,257, a two-layer random transformer instead of GPT-2, and 4000 samples. At that scale the sampling noise floor is around 0.02 TV, so a threshold of 0.05 catches a real bug while tolerating chance.
The comment in the test states the expected noise level explicitly. That single line is the difference between a test with a threshold someone tuned until it passed and a test with a threshold someone derived. It is the most sophisticated piece of testing in the repository.

11.5 Two caches, two rollbacks

The draft model needs its own KV cache, its own `BlockManager`, and its own `Sequence` mirroring each target sequence. The project calls these *shadows*.

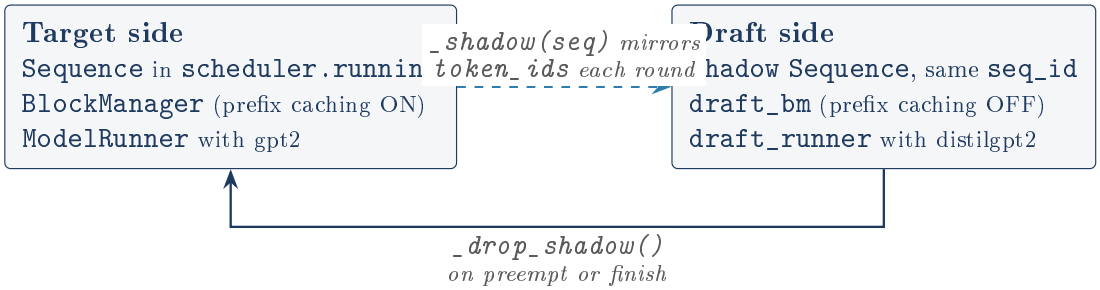


Figure 15: The shadow arrangement. A rejection rolls back *both* block tables (`block_manager.truncate` and `draft_bm.truncate`) plus the prefix hashes on the target side.

The rollback is the trickiest sequence of assignments in the codebase:

```
base_len = len(seq) - k              # length before drafts were appended
target_probs = target_probs_from_logits(logits[i], seq.params, seq.token_ids)
result = accept_reject(draft_tokens[seq.seq_id], draft_probs[seq.seq_id],
                       target_probs, seq.generator, self.spec_stats)
new_len = base_len + result.num_accepted
seq.token_ids = seq.token_ids[:new_len] + [result.tokens[-1]]
seq.num_computed = new_len
self.block_manager.truncate(seq, len(seq.token_ids))
shadow.token_ids = list(seq.token_ids)
shadow.num_computed = min(shadow.num_computed, new_len)
self.draft_bm.truncate(shadow, len(shadow.token_ids))
```

Listing 17: llm_engine.py, _speculative_decode: the rollback

Note `seq.num_computed = new_len` while `len(seq.token_ids)` is `new_len + 1`: the invariant `num_computed == len - 1` is restored exactly, so the next ordinary step sees a normal-looking sequence. Every speculative round leaves the world indistinguishable from a plain decode that happened to emit several tokens.

The draft cache's capacity invariant is false, and breaking it kills the server (verified)

`docs/ARCHITECTURE.md` states: "The draft cache runs without prefix caching so its block usage is always a subset of the target's; one capacity check then covers both caches." A matching comment sits in `llm_engine.py`. The scheduler checks only `self.block_manager` (the target's), and `_shadow` raises a bare `RuntimeError("draft KV cache exhausted")` if the draft cannot allocate.

The reasoning is about *retention*: the draft returns freed blocks to the free list immediately rather than parking them in an evictable pool, so it never hoards. That much is true. But it omits *sharing*, which runs the other way. The target's `fork()` path (and its prefix cache) let several sequences point at the same physical blocks; the draft allocates a private copy of every block for every shadow. So for sequences with a shared prefix the target's physical usage is *lower* than the sum over sequences while the draft's is exactly the sum. The draft therefore needs *more* blocks than the target, the opposite of the claimed subset relation.

Note this does not even depend on prefix caching: `fork()` shares blocks unconditionally, so the stated *reason* for the invariant ("runs without prefix caching") does not deliver the invariant even on its own terms.

This was driven, not deduced. An engine with `num_blocks=12`, a 63-token shared prompt and `n=4` completions, target and draft both distilgpt2:

```
prompt tokens: 63
target blocks total: 12 | draft blocks total: 12
requested n=4 -> 4 sequences
  step 1: target_used= 4  draft_used= 0
RESULT: RuntimeError at step 2: draft KV cache exhausted
```

Step 1 shows the sharing working exactly as designed: four sequences occupy four blocks between them. The target-side check then passes comfortably, and the draft needs $4 \times 4 = 16$ blocks against 12 and dies. The capacity check that is documented as covering both caches did not cover the draft.

At the default configuration arithmetic saves it by luck rather than by the invariant: `max_batch_size=8` times `max_model_len=1024` is 8192 tokens, exactly `num_blocks=512` times `block_size=16`, so the draft cannot overflow even with zero sharing. Raise `max_batch_size` and the margin is gone. The project's own benchmark harness starts its server with `--max-batch-size 16` (`start_server` in `bench.py`), double the safe bound, though it never combines that with a draft model. No test covers a draft engine under KV pressure with shared prefixes.

Any exception in `step()` permanently kills the server (verified)

This is the finding that turns the previous one from an accounting error into an outage, and it is independent of it.

`AsyncEngine._run_loop` has **no exception handling whatsoever**:

```
async def _run_loop(self):
    while True:
        if not self.engine.has_work:
            self._wake.clear()
            await self._wake.wait()
        outputs = await asyncio.to_thread(self._locked_step)   # raises -> task dies
        ...
```

The loop runs as a bare `asyncio.Task` created in `start()`. Nothing awaits it, nothing attaches a done-callback, and nothing inspects its exception. So when `step()` raises, the task dies silently and the server keeps accepting requests it will never answer. Every client hangs forever, on a healthy looking process whose `/healthz` still returns `{"status": "ok"}`.

Driven with the same tight draft configuration, through `AsyncEngine`:

```
submitted, seq_ids: [0, 1, 2, 3]
RESULT: *** STREAM HUNG *** (25s timeout, no tokens ever arrived)
background step-loop task done?  True
step-loop died with: RuntimeError('draft KV cache exhausted')
second request: *** ALSO HUNG *** -> server is permanently dead
```

Note the last line: it is not only the offending request that hangs. A subsequent, entirely innocent `"Hello there"` request hangs too, because the loop that would have served it no longer exists. One bad request permanently bricks the process until someone restarts it.

The fix is routine (wrap the step in `try/except`, fail the affected sequences, keep looping) and its absence is the single most consequential gap in the project: every other bug in this document degrades quality or wastes memory, and this one converts *any* of them into a total outage. It is worth being the first thing volunteered about the async layer, because "one coarse lock, carefully reasoned" and "no error handling at all" sit in the same 82-line file.

One slow sequence throttles the whole batch's lookahead

```
k = min(self.config.num_speculative_tokens,
        min(self.config.max_model_len - len(s) for s in seqs),
        min(s.num_prompt_tokens + s.params.max_tokens - len(s) for s in seqs))
```

`k` is a *minimum across the batch*, because the verify pass must be one rectangular `[B, k+1]` forward. So a single sequence one token from its `max_tokens` limit drags `k` to 1 for everybody,

and if it hits zero the whole batch silently falls back to `self._decode(seqs)`. With eight sequences finishing at staggered times, the batch spends much of its life at a reduced `k`. This is a genuine consequence of the rectangular-batch design and it is not mentioned anywhere in the documentation.

`num_speculative_tokens` applies even with no draft model

`EngineConfig.num_speculative_tokens` defaults to 4 whether or not `draft_model` is set, and `add_request` uses it unconditionally:

```
worst_blocks = (max_len + self.config.num_speculative_tokens
                + self.config.block_size - 1) // self.config.block_size
```

So a plain engine reserves headroom for four speculative tokens it will never draft when deciding whether a request can fit at all. The effect is negligible (it can reject a request that is within one block of the absolute maximum) but it is a config value leaking into a code path that its feature is switched off for. Meanwhile `Scheduler.lookahead` *is* correctly gated on `draft_model` being set. The two places disagree about the same question.

12 The async engine and the HTTP server

12.1 Why a wrapper exists at all

Jargon: Event loop, blocking, `asyncio`

Python's `asyncio` runs many tasks on one thread by having each task voluntarily yield while it waits for something slow, like a network read. This works only if no task ever does a long stretch of CPU work without yielding: that would *block* the loop and freeze every other connection. A transformer forward pass is exactly such a stretch.

```
async def _run_loop(self):
    while True:
        if not self.engine.has_work:
            self._wake.clear()
            await self._wake.wait()          # sleep until a request arrives
        outputs = await asyncio.to_thread(self._locked_step)
        for out in outputs:
            queue = self._queues.get(out.seq_id)
            if queue is not None:
                queue.put_nowait(out)

def _locked_step(self):
    with self._lock:
        return self.engine.step()
```

Listing 18: async_engine.py, the whole trick

`asyncio.to_thread` pushes the blocking `step()` onto a worker thread, so the event loop stays free to accept connections and flush SSE frames while the model computes. The `_wake` event means an idle server does not spin.

Why one coarse lock is the right call here, and when it stops being

A single `threading.Lock` serializes `step()`, `add_request()` and `abort()`. This is as coarse as locking gets and it is the correct choice for this project: on CPU the model forward dominates step time so completely that lock contention is unmeasurable, and one lock is the version a

person can reason about with certainty.
It stops being right the moment you want to overlap prefill with decode, or pipeline the sampler behind the next forward, both of which require the engine's state to be mutable by two things at once. The README lists this correctly under "what I'd do differently" and gives the right reason. Volunteering "here is the simple thing, here is exactly when it breaks" is a better answer than either defending it as optimal or apologizing for it.

Jargon: GIL (Global Interpreter Lock)

CPython permits only one thread to execute Python bytecode at a time. Threading therefore does not speed up Python code. It does help here because PyTorch releases the GIL during tensor operations: while `step()` is inside a matrix multiply, the event loop thread genuinely runs. The design depends on this, and it is nowhere stated in the code.

12.2 The request path

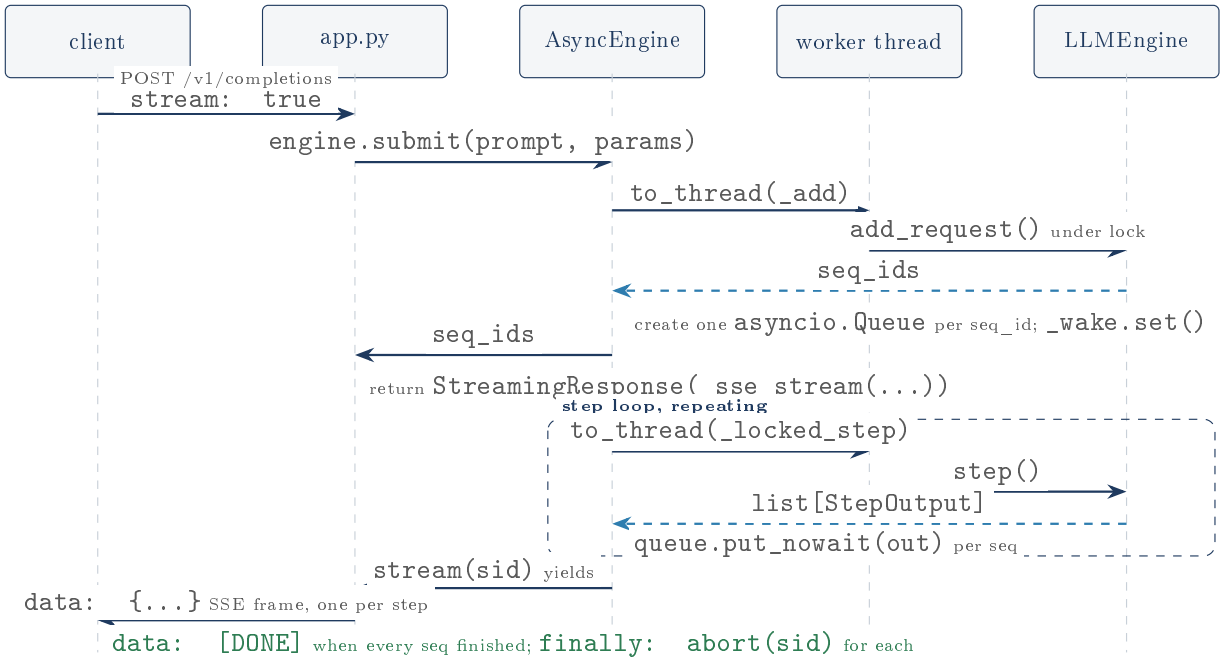


Figure 16: One streaming request. The engine never learns what HTTP is; it puts `StepOutput` objects into queues, and `app.py` turns those into SSE frames.

Jargon: SSE (Server-Sent Events)

A way for a server to keep an HTTP response open and push text at the client as it becomes available, each chunk prefixed `data:` and terminated by a blank line. It is how a chatbot's answer appears word by word instead of all at once. Simpler than a WebSocket and one-directional, which is all that is needed.

The fan-out for `n > 1` merges several per-sequence queues into one output queue so that the SSE frames interleave as they are produced rather than completing one choice before starting the next:

```
async def one(index, sid, out_q):
    async for out in engine.stream(sid):
        await out_q.put((index, out))
```

```
    await out_q.put((index, None))          # sentinel: this choice is done

out_q = asyncio.Queue()
tasks = [asyncio.create_task(one(i, sid, out_q)) for i, sid in enumerate(seq_ids)]
live = len(seq_ids)
while live > 0:
    index, out = await out_q.get()
    if out is None:
        live -= 1
        continue
    if await raw.is_disconnected():
        break
    yield f"data: {json.dumps(payload)}\n\n"
```

Listing 19: app.py, _sse_stream fan-in

The `finally` block is the part that matters for a server that must survive its clients: it cancels the fan-in tasks and aborts every sequence, so a client that hangs up mid-generation does not leave the engine computing tokens nobody will read.

12.3 The API surface

Endpoint	Method	Purpose
`/v1/completions`	POST	the OpenAI completions subset, streaming or not
`/v1/models`	GET	reports the single loaded model
`/healthz`	GET	liveness, used by the benchmark's `start_server`
`/metrics`	GET	engine counters, the observability surface

`/metrics` is worth calling out because it exposes precisely the internals this document has been describing:

- `num_free_blocks`, `num_total_blocks`: KV cache pressure
- `prefix_cache_queries`, `prefix_cache_hit_tokens`: is the prefix cache earning its keep
- `cow_copies`, `evictions`: how much sharing is breaking down
- `preemptions`: is the engine thrashing
- `speculative_acceptance_rate`: is the draft model worth its cost

Someone operating this server could actually diagnose it, which is more than many production systems offer.

`/healthz` reports liveness it has not checked

Following directly from the step-loop finding: `healthz` returns a hardcoded `{"status": "ok"}` without consulting the engine at all. It cannot distinguish a healthy server from one whose step loop died twenty minutes ago. Since the whole point of a liveness endpoint is to let an orchestrator restart a wedged process, and this project's step loop can wedge, the one mechanism that would have recovered the outage automatically is the one that reports everything is fine. Checking `engine._task.done()` would be a two-line fix.

The OpenAI compatibility is a thin subset, and `logprobs` is a lie of omission

Every response includes `"logprobs": None`, which is an OpenAI field the engine does not implement; it emits the key and always nulls it. Also absent: `/v1/chat/completions` (the endpoint essentially every client actually uses), `echo`, `best_of`, `presence_penalty`,

`frequency_penalty`, and a `usage` token-count block. The README says "OpenAI subset", which is accurate. Just be ready for "so can I point the OpenAI SDK at it?": the answer is yes for `client.completions.create`, no for `client.chat.completions.create`.

13 What was actually verified here

Everything in this section was executed on this machine while writing this document. Nothing in it is taken from the README on trust.

13.1 The declared dependencies do not run the project

The very first thing attempted was `pytest`, and it failed. Not on a subtle assertion: on *loading a model at all*.

```
E  TypeError: GPT2LMHeadModel.__init__() got an unexpected keyword argument 'dtype'
...
4 failed, 4 passed, 1 warning, 9 errors in 1008.34s
```

`requirements.txt` declares a version range the project cannot run in

`engine/model.py` calls `AutoModelForCausalLM.from_pretrained(model_name, dtype=torch.float32)`. The `dtype` keyword is recent: Hugging Face spent years spelling it `torch_dtype` and only accepted `dtype` later. `requirements.txt` declares:

```
transformers>=4.44,<6
```

This was bisected against the real package index on this machine:

transformers	In declared range?	Result
4.51.3	yes	`TypeError`, no model loads
4.55.4	yes	`TypeError`, no model loads
4.56.0	yes	works
5.14.1	yes	works

So the true floor is **4.56.0**, and the declared floor is 4.44. Every version from 4.44 to 4.55.4, which is roughly two years of releases and the larger part of the declared range, gives a clone of this repository that cannot load a single model. A reviewer who runs `pip install -r requirements.txt` today gets a working resolution only because pip picks the newest; a reviewer with a pinned or cached environment, or anyone reading this in a year against `<6`, may not. The machine used for this document had 4.51.3 installed system-wide, which is exactly how the problem surfaced.

The fix is one character short of trivial (`transformers>=4.56,<6`). The finding is not the severity, it is the category: the project's headline claim is that it was *verified by actually running it*, and its declared environment is not the environment it was verified in. Everything below was therefore run against transformers 5.14.1 in an isolated virtualenv, not against the declared range.

13.2 The test suite

Module	Tests	What it covers
`test_block_manager.py`	15	allocation, COW, prefix cache, LRU, truncate
`test_scheduler.py`	6	admission, preemption, the 300-step churn test
`test_sampling.py`	7	greedy, top-k, top-p, penalties, determinism
Total	**28**	**28 passed in 0.76s**

Table 2: The model-free suites, run against the system interpreter. That the allocator, the scheduler and the sampler can be fully exercised in under a second without loading a single weight is a direct dividend of the `BlockManager`-holds-no-tensors decision, and it is why these 28 passed on an environment where nothing model-backed could even start.

Re-run in an isolated virtualenv on transformers 5.14.1, where the model-backed tests can actually load a model, the **entire suite passes**:

```
55 passed, 2 warnings in 897.56s (0:14:57)
```

The code is correct; only the declared environment is wrong

This is the fair reading, and it matters. All 55 tests pass: the logit parity against Hugging Face, the paged-versus-plain attention check, the 300-step churn property test, the 4000-sample speculative distribution tests, the concurrent streaming API tests. Nothing in the engine is broken.

The dependency finding above is therefore about packaging, not about the code. It would be unfair to read the initial red wall of errors as "the project does not work". It works; it just does not declare the environment it works in. Those are different criticisms and only the second one is true.

13.3 The `seq_states` leak, confirmed

The leak described earlier was not inferred from reading. It was driven: twenty sequential `generate()` calls against a `distilgpt2` engine, then a read of the engine's internals.

```
generates issued      : 20
len(engine.seq_states): 20
scheduler.running     : 0
scheduler.waiting     : 0
free blocks before/now: 128 128
shadows               : 0
sample retained state : seq_id=0 emitted_text=' "I\'m not sure what'
```

Every other structure cleaned up perfectly: the scheduler's queues are empty, the block manager is back to all 128 blocks free, no shadows remain. Only `seq_states` still holds all twenty entries, each with the finished request's complete generated text. The behaviour matches the source exactly: `_finish()` has no `seq_states.pop`. The contrast is the interesting part, since the memory management the project is *about* is impeccable, and the leak is in the incidental bookkeeping beside it.

13.4 Benchmarks: what the README claims and what this run observed

Why these numbers could be re-run at all

This is unusually lucky and worth stating. The committed JSON in `benchmarks/results/` records the machine each number came from, and it is an 11th Gen Intel i7-1185G7 with 8 cores running the same Linux kernel as the machine this document was built on. The cached `gpt2` and `distilgpt2` weights are present locally. So the benchmarks are not being taken on trust or approximated: they were re-executed on the same hardware, and the results below are what this machine actually printed.

13.4.1 Conditions of the re-run, stated up front

Three things differed from the committed run of 2026-07-06, and they must be on the table before any number is:

- **Library versions.** The re-run used `torch 2.10.0+cu128` and `transformers 5.14.1`. The original's versions are *unknown*, for a reason that is itself a finding (below). Note also that `+cu128` is a CUDA build being run on CPU, whereas the README's own setup instructions install the CPU-only wheel. That alone can move CPU kernel performance.
- **Background load.** The machine had a desktop session and several other processes active. The heavy competitors were waited out (the run started at a 1-minute load average of 4.77), but this was not a quiet benchmarking host, whereas the original may have been.
- **Thread count.** `torch.get_num_threads()` reports **4**, not 8. The i7-1185G7 has 4 physical cores and 8 logical threads; PyTorch defaults to physical cores. The README describes the machine as "8 threads", and the JSON's `cores: 8` is `os.cpu_count()`, which counts logical processors. The compute actually ran on 4.

The benchmark harness does not record the versions its numbers depend on

`machine_info()` in `bench.py` captures CPU model, core count, platform, Python version and `"device": "cpu"`. It does not capture the `torch` version, the `transformers` version, or `torch.get_num_threads()`.
Those are precisely the variables a throughput number depends on. The committed JSON therefore records enough to identify the *machine* and not enough to reproduce the *measurement*, which is the whole purpose of committing it. It is the reason the discrepancy below cannot be fully attributed: the experiment that would settle it (re-run on the original's library versions) is not recoverable from what was recorded. Three extra lines in `machine_info()` would have prevented that permanently.

13.4.2 What the re-run measured

Measurement	README / JSON	Re-run	Re-run / orig
Throughput, concurrency 1 (tok/s)	44.69	32.90	0.74x
Throughput, concurrency 4 (tok/s)	83.80	73.53	0.88x
Throughput, concurrency 8 (tok/s)	113.52	99.19	0.87x
Throughput, concurrency 16 (tok/s)	108.47	108.88	1.00x
HF `generate()` sequential (tok/s)	40.04	31.89	0.80x
TTFT mean, concurrency 1 (s)	0.069	0.089	
TTFT mean, concurrency 4 (s)	0.266	0.289	
TTFT mean, concurrency 8 (s)	0.541	0.604	
TTFT mean, concurrency 16 (s)	1.266	1.149	

Table 3: Absolute throughput came in 13% to 26% below the committed numbers, which is what a contended, differently-versioned host should do. Every *shape* the README describes reproduced: throughput climbing with concurrency and flattening by 16, TTFT growing roughly linearly with queue depth.

Absolute numbers are the least interesting thing here, because they are the most environment-dependent. The README makes *ratio* claims, and ratios are what should survive. They did:

Claim the README actually makes	README	Re-run	Verdict
Continuous batching, concurrency 1 to 8	2.54x	**3.01x**	reproduced, stronger
Engine at 8 concurrent vs HF sequential	2.84x	**3.11x**	reproduced, stronger
Speculative decoding vs plain	0.72x	**1.14x**	**did not reproduce**
Speculative acceptance rate	0.5404	**0.5404**	exact, to 4 decimals

The two headline claims reproduced, and the re-run is kinder than the README

The task this document was written for asked specifically about "113.5 tokens/s and 2.8x over a baseline". The verdict on both:

113.5 tok/s did not reproduce as an absolute number: this machine measured 99.19 tok/s at the same concurrency 8, about 13% lower, on a contended host with unknown version drift. The number is plausible and clearly of the right order, but it should be quoted as "about 100 to 115 tok/s on a laptop CPU depending on conditions", not as a constant.

The 2.8x reproduced, and came out at 3.11x. The engine beat sequential Hugging Face by *more* than the README claims, because contention hurt the one-at-a-time baseline more than it hurt the batched engine, which is exactly the effect continuous batching exists to produce. The same is true of the 2.5x batching claim, which measured 3.01x.

So the README's two headline performance claims are conservative on this machine. That is the right direction for a claim to be wrong in, and it is consistent with the pattern of everything else in this project: nothing is inflated.

The speculative decoding result reversed sign, and this is the real discrepancy

The README's most distinctive claim is a *negative* one, and it is the one that did not hold:

> "Speculative decoding is correct (identical greedy output, 54% of drafts accepted) but ~0.7x slower here ... It stays in the codebase as a correctness-verified feature, not a CPU speedup."

The re-run measured **1.14x faster**, not 0.72x slower. Broken out:

	README	Re-run	
plain, wall time (s)	5.728	8.105	plain got *much* worse
speculative, wall time (s)	7.958	7.111	speculative got *better*
tokens drafted	322	322	identical
tokens accepted	174	174	identical
acceptance rate	0.5404	0.5404	identical
speedup	0.72x	**1.14x**	sign flip

First, what this is not. It is not a correctness problem, and the evidence is unusually clean: 322 drafted and 174 accepted, in both runs, to the token. Greedy decoding is deterministic, so the algorithm made bit-for-bit identical decisions six weeks and two library versions apart. The engine's determinism claim is confirmed about as strongly as it could be. Only the *clock* disagreed.

What changed. Under uniform slowdown a ratio is preserved, and this ratio was not, so the effect is differential: plain decode degraded 41% while speculative *improved* 11%. The coherent explanation is per-forward overhead. Plain decode does many tiny forwards (one token position each); speculative does far fewer, fatter ones (the draft's k, then one target forward over k+1=5 positions). Anything that raises fixed per-forward cost, CPU contention, a different torch build, a CUDA-enabled wheel dispatching on CPU, penalizes the many-small-forwards strategy and barely touches the fewer-large-forwards one.

The same mechanism explains the otherwise odd row in the table above: concurrency 1 fell to 0.74x while concurrency 16 held at 1.00x. Large batches amortize per-forward overhead; batch-of-one does not. The two anomalies have one cause.

Why this matters more than a number. The README draws a firm conclusion from its measurement ("not a CPU speedup") and the conclusion is not stable: the sign of that result depends on the host. Ironically, the README's own reasoning predicts this. It says speculative decoding "pays off when the target forward dominates draft cost", and per-forward overhead is one of the things that shifts that balance. The project reasoned correctly and then over-committed to one sample of a quantity it had itself identified as conditional.

The honest position is narrower and stronger than either the README's or a naive "it is actually faster": on this hardware, speculative decoding with `gpt2`/`distilgpt2` at k=4 sits near break-even, and which side of it you land on depends on the software environment. Two measurements exist, 0.72x and 1.14x, and neither is the property of the algorithm. What is stable, and what the project can defend without qualification, is the acceptance rate and the exact output equivalence.

The committed JSON was left exactly as it was

Re-running `bench.py all` overwrites `benchmarks/results/*.json`. The originals were restored byte-for-byte afterwards, so the repository's committed measurements are untouched and this document's comparison is against them as recorded. The re-run's numbers live only in this document. If the owner wants the new figures committed, that is a deliberate decision to make separately, and it should not be made from a contended host.

14 Consolidated honest assessment

14.1 What is genuinely strong

1. **The correctness evidence is real and layered.** Logit parity against Hugging Face at `1e-3`, a paged-versus-plain attention check at `1e-4`, a 300-step property test on block accounting, and

a 4000-sample total-variation test on the speculative distribution. The project does not assert its correctness; it demonstrates it at four different levels of abstraction.

2. **The BlockManager holds no tensors.** This one decision is why the hardest logic in the codebase is testable in under a second, and it is the thing most worth stealing from this project.

3. **The honesty is unusual and is an asset.** A README that reports its own headline feature going 0.72x, and explains why, is more persuasive than one reporting a 2x speedup. The absent vLLM comparison is stated as absent, with the reason (no GPU) rather than a fabricated number.

4. **The numbers in the README are faithful to the JSON.** Every figure checked (44.7, 83.8, 113.5, 108.5, 40.0, 32.2, 0.54, the 2.5x, the 2.8x, the 0.7x) transcribes or divides correctly from `benchmarks/results/`. There is no rounding in the flattering direction.

14.2 The findings, ranked by what they would cost in an interview

The top three were all found by *running* the project rather than reading it, which is itself the lesson: the code reads well enough that none of them are visible from the source alone.

1. **No error handling in the step loop, so any `step()` exception permanently bricks the server** (verified: a second innocent request hangs too, while `/healthz` still reports ok). This ranks first because it is a severity multiplier: it converts every other bug in this list, and every one not yet found, into a total outage rather than a bad response.

2. **The draft cache's capacity invariant is false and reachable** (verified: `RuntimeError: draft KV cache exhausted` on the second step of an `n=4` shared-prompt request). Worse than a bug, it is a *documented* invariant in `docs/ARCHITECTURE.md` that the code does not have, and the stated reason for it ("runs without prefix caching") does not imply it even in principle, because `fork()` shares blocks regardless. Being asked "why is one capacity check enough?" and answering from the architecture document would be answering wrongly. Combined with item 1, it is a remotely triggerable permanent outage.

3. `requirements.txt` **declares a range the project cannot run in** (verified by bisection: needs `transformers>=4.56`, declares `>=4.44`). Cheap to fix, and awkward for a project whose central claim is that nothing is asserted that was not verified by running it.

4. `seq_states` **leaks for every completed request** (verified: 20 entries retained after 20 finished generations, each holding the full output text). A memory leak in the long-running-service part of a project about long-running services.

5. **Seeded `n > 1` determinism is load-dependent** because the no-batch-room path burns a generator draw before requeueing. A documented guarantee with a hole and no test.

6. **The TTFT curve is a readout of unbatched prefill**, not a general property of continuous batching. The README treats it as expected without connecting it to its own design note two sections away.

7. **The attention is a gather-into-dense, not a paged kernel.** Already disclosed in the README, so it costs nothing if volunteered and a lot if discovered.

8. **The speculative result's sign is environment-dependent** (measured 1.14x here against the README's 0.72x, with a bit-identical acceptance rate). The README states a conditional result as a settled one. Related: **the harness records no library versions**, so the discrepancy cannot be attributed and the committed numbers cannot truly be reproduced.

9. **The detokenizer is** $O(n^2)$ in output length, invisible at the benchmark's 32 tokens.

10. **Smaller items:** `_match_prefix` computed three times per admission; `EOS_TOKEN_ID` hardcoded rather than read from the tokenizer; `num_speculative_tokens` applied with no draft model; `slot_mapping` rebuilding whole-context Python lists every step; just-admitted prompts pre-emptible in the same step; prefix cache trusting SHA-256 without verification; `/healthz` reporting liveness it never checks.

The pattern behind the top four

Every one of them lives in the space *around* the algorithms rather than in them. The paged cache, the scheduler, the accept/reject sampler: all correct, all well tested, all defensible in detail. The failures are in dependency declaration, process lifetime, error propagation and cross-component invariants, which is precisely the set of things unit tests do not reach and a short benchmark run cannot reveal.
That is a genuinely useful thing to be able to say about one's own project: the hard part was done well, and the gaps are in the boring parts. It is also the honest answer to "what would you do next", ahead of the GPU kernel.

14.3 What the benchmarks do and do not support

The "2.8x over baseline" is honest but not apples-to-apples

The claim compares 113.52 tok/s (this engine, 8 concurrent) against 40.04 tok/s (HF `generate()`, sequential). Both numbers are real and both are in the committed JSON. But they come from *different harnesses*: `cmd_sweep` drives a real server process over HTTP with SSE parsing, while `cmd_baseline` calls `ref.generate()` in-process with no serialization at all. The direction of the bias favours the baseline: the engine is carrying HTTP, JSON, SSE and asyncio overhead that the baseline does not, so the true engine-versus-HF gap is if anything *wider* than 2.8x. The comparison is therefore conservative, which is the right way to be wrong. It is still not a controlled comparison, and the honest framing is "2.8x end-to-end including our serving overhead against a bare in-process loop", which is a more interesting claim anyway. Note also that HF `generate()` sequentially is a weak baseline by construction: the fair strong baseline is HF with its own batching, which is not measured.

The token counter in the sweep counts SSE events, not tokens

`_stream_one` in `bench.py` does `tokens += 1` per `data:` line rather than counting `new_token_ids`. For plain decoding one step emits one token so one event is one token, and the totals check out exactly ($8 \times 32 = 256$). But it is a proxy that happens to be right, not a measurement. If the sweep were ever run with `--draft-model` set, one event would carry up to $k + 1$ tokens and the reported throughput would be *understated by up to 5x* with no warning. A latent trap in the harness for whoever runs the GPU comparison the README calls future work.

The speculative benchmark measures a warm prefix cache

`cmd_speculative` runs `run(plain)` once to warm up and then measures a second `run(plain)` over *the same four prompts*, with prefix caching on by default. The measured run therefore gets its prompts back from the prefix cache almost for free. The same is true of the speculative engine, so the comparison between them is fair, which is what the benchmark is for. It does mean the 44.69 tok/s figure is a warm-cache number, and *that same figure* is reused in the README's baseline table as "this engine, 1 concurrent request", where it is being compared against a cold HF baseline. It is a small effect on a 32-token workload, and it points the same conservative direction. Worth knowing before someone asks whether the caches were warm.

14.4 The one thing this project is missing

The README says it plainly and it is correct: there is no comparison against a production engine, because vLLM needs a GPU and this machine has none. That is the right call. Fabricating the number would be worse than not having it.

But it means the project can demonstrate that its *mechanisms* work and cannot demonstrate that they work *well*. Everything measured here says "continuous batching gives 2.5x on 8 CPU threads". Nothing measured here says anything about how this design would behave where these designs actually live. The project knows this, states it, and lists it as the obvious next step, which is the only defensible position available without hardware.

15 Glossary

Term	Meaning in this project
Token	An integer id for a chunk of text. GPT-2 has 50,257.
Logits	50,257 raw scores for what the next token should be.
KV cache	Stored keys and values for past tokens, so they are computed once.
Block	A fixed-size chunk of KV cache holding `block_size` (16) tokens.
Slot	A single token's place in the cache: `block_id * block_size + offset`.
Block table	A sequence's list of block ids: its page table.
Prefill	The first forward over a whole prompt. Compute-bound.
Decode	Each subsequent one-token-per-sequence forward. Memory-bound.
Continuous batching	Re-deciding the batch at every token boundary.
Preemption	Evicting a running sequence's blocks under KV pressure.
COW	Copy-on-write: share a block until someone writes, then copy.
Prefix caching	Reusing cached blocks for a prompt prefix already seen.
Draft / target	The small guessing model and the real model that verifies.
Acceptance rate	Fraction of drafted tokens the target agreed with.
TTFT	Time to first token: how long a user waits before anything appears.
Shadow	The draft side's mirror of a target sequence.
SSE	Server-Sent Events: the streaming HTTP response format.
TV distance	Total variation: how far apart two distributions are.